ASESII 24A02 Final Project

Predict noise level generated by NACA airfoil blade sections

Group 02
Nguyen Thi Yen Nhi (24125071)
Nguyen Khang Thinh (24125104)
Nguyen Van Tinh (24125106)
Nguyen Le Quoc Huy (24125100)

2026-08-23

Contents

Authorship and contribution statement **2**

Abstract **2**

Reproducible setup **3**

Shared helper functions 3

1 Problem 1. Prediction design and leakage control **6**

1.1 1A. Target and predictors 6

1.2 1B. Split and preprocessing plan 8

1.3 1C. Data exploration 12

2 Problem 2. OLS, Ridge, and Lasso **18**

2.1 2A. OLS baseline 18

2.2 2B. Ridge regression 25

2.3 2C. Lasso regression 28

2.4 2D Controlled comparison and locked core model 31

2.5 2E. Perturbation or stability check 32

3 Problem 3. Mathematical mechanisms **35**

3.1 3A. Ridge and conditioning 36

3.2 3B. Lasso optimality and soft thresholding 37

3.3 3C. Connect algebra to evidence 41

4 Problem 4. Elastic net and a neural-feature bridge **41**

4.1 4A. Research question and source map: L_1 penalties and sparse weights. 41

4.2 4B. Elastic net on original predictors 43

4.3 4C. Fixed ReLU features 46

4.4 4D. Locked the analysis and evaluate once 51

4.5 4E. Deep-learning connection and limitations 53

5 Problem 5. Report quality and reproducibility **55**

5.1 5A. Reproducible submission 55

5.2 5B. Statistical coummunication 55

5.3 5C. AI verification record 56

Preflight and session information **57**

Authorship and contribution statement

Student	ID	Main contributions	Verification performed
Nguyen Thi Yen Nhi	24125071	1A (Data analysis), 2A (OLS), 4E (Question Explanation)	Verify AI texts
Nguyen Khang Thinh	24125104	1C (EDA), 2C (Lasso), 4B (Elastic Net), 4C (ReLU features), 4D (Holdout Eval)	Verified code & AI logs
Nguyen Van Tinh	24125106	1B (Data split), 3A (Ridge), 3B (Lasso), 3C (Algebra Evidence)	Verify mathematical problems
Nguyen Le Quoc Huy	24125100	2B (Ridge), 2D (Model comparison), 2E (Stability Check), 4A (Paper research)	Verify the project workflow & code

Abstract

This study aims to predict pre-operative log prostate-specific antigen (`lpsa`) from clinical measurements, comparing models on prediction error, coefficient stability, and interpretability under multicollinearity. We evaluated OLS, Ridge, Lasso, and Elastic Net regressions using five-fold cross-validation on original linear predictors and fixed random ReLU features. Cross-validation initially favored Lasso (CV-MSE = 0.5790) for its sparse selection of five variables. However, holdout evaluation revealed that the OLS model achieved the lowest test RMSE (0.6713) and MAE (0.5736). Furthermore, original linear features consistently outperformed the nonlinear ReLU representations across all models. Despite the OLS superiority in pure holdout accuracy, we recommend the lasso-dominant elastic net ($\alpha = 0.9$) for clinical deployment, as it provides a parsimonious, highly interpretable model that isolates key biological drivers of PSA with only a modest trade-off in error.

Reproducible setup

```
required_packages <- c("tidyverse", "glmnet", "broom", "knitr", "car")
missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]
if (length(missing_packages)) {
  stop("Install the required packages before rendering: ",
       paste(missing_packages, collapse = ", "))
}

library(tidyverse)
library(glmnet)
library(broom)
library(knitr)
library(car)

# `params` exists only while knitting; the fallback
# keeps interactive chunk runs working.
has_params <- exists("params") && !is.null(params$group_number)
group_number <- if (has_params) as.integer(params$group_number) else 4L
if (length(group_number) != 1L || is.na(group_number) || group_number < 1L) {
  stop("Set params$group_number to your assigned positive group number.")
}

seeds <- list(
  split = 240200L + group_number,
  cv = 240300L + group_number,
  features = 240400L + group_number
)
```

Shared helper functions

```
find_exam_data <- function(filename) {
  input <- knitr::current_input(dir = TRUE)
  input_dir <- if (length(input) && nzchar(input)) dirname(input) else getwd()
  candidates <- c(
    file.path(input_dir, "data", filename),
    file.path(input_dir, "..", "data", filename),
    file.path(getwd(), "data", filename)
  )
  existing <- candidates[file.exists(candidates)]
  if (!length(existing)) stop("Cannot locate ", filename, " by a relative path.")
  hits <- unique(normalizePath(existing, winslash = "/", mustWork = TRUE))
  if (length(hits) > 1L && length(unique(unname(tools::md5sum(hits)))) > 1L) {
```

```
    stop("Multiple nonidentical copies of ", filename, " were found.")
  }
  hits[[1L]]
}

split_rows <- function(n, train_prop = 0.80, seed) {
  stopifnot(n >= 10L, train_prop > 0, train_prop < 1)
  set.seed(seed)
  train <- sort(sample.int(n, floor(train_prop * n), replace = FALSE))
  list(train = train, test = setdiff(seq_len(n), train))
}

fit_scaler <- function(x, tol = 1e-12) {
  x <- as.matrix(x)
  center <- colMeans(x)
  centered <- sweep(x, 2L, center, "-")
  scale <- sqrt(colMeans(centered^2))
  keep <- is.finite(scale) & scale > tol
  if (!any(keep)) stop("No nonconstant training predictor remains.")
  list(
    center = center[keep],
    scale = scale[keep],
    keep = keep,
    dropped = colnames(x)[!keep]
  )
}

apply_scaler <- function(x, prep) {
  x <- as.matrix(x)[, prep$keep, drop = FALSE]
  x <- sweep(x, 2L, prep$center, "-")
  sweep(x, 2L, prep$scale, "/")
}

make_foldid <- function(n, k = 5L, seed) {
  if (k < 3L || k > n) stop("Require 3 <= k <= number of training rows.")
  set.seed(seed)
  sample(rep(seq_len(k), length.out = n))
}

fit_cv_glmnet <- function(x, y, alpha, foldid) {
  glmnet::cv.glmnet(
    x = x,
    y = y,
    family = "gaussian",
    alpha = alpha,
    foldid = foldid,
    standardize = FALSE,
    intercept = TRUE,
```

```
    type.measure = "mse"
  )
}

score_regression <- function(y, prediction) {
  y <- as.numeric(y)
  prediction <- as.numeric(prediction)
  if (length(y) != length(prediction) || any(!is.finite(c(y, prediction)))) {
    stop("Metrics require equal-length finite vectors.")
  }
  c(RMSE = sqrt(mean((y - prediction)^2)),
    MAE = mean(abs(y - prediction)))
}

count_nonzero <- function(fit, s = "lambda.min", tol = 1e-8) {
  b <- as.matrix(stats::coef(fit, s = s))
  slopes <- b[rownames(b) != "(Intercept)", 1L]
  sum(abs(slopes) > tol)
}

safe_condition_numbers <- function(x, lambda, tol = 1e-10) {
  eigenvalues <- eigen(
    crossprod(x) / nrow(x),
    symmetric = TRUE,
    only.values = TRUE
  )$values
  largest <- max(eigenvalues)
  smallest <- max(min(eigenvalues), 0)
  cutoff <- tol * max(1, largest)
  c(
    gram = if (smallest <= cutoff) Inf else largest / smallest,
    ridge = (largest + lambda) / (smallest + lambda)
  )
}
analysis_model <- function(x, y, alpha, foldid) {
  model <- fit_cv_glmnet(x = x, y = y, alpha=alpha, foldid = foldid)
  list(
    model = model,
    lambda_min = model$lambda.min,
    mse_min = model$cvm[model$lambda == model$lambda.min],
    cond_min = safe_condition_numbers(x, lambda = model$lambda.min)[2],
    coef_min = coef(model, "lambda.min"),
    coef_min_norm = sqrt(sum(coef(model, "lambda.min")[-1]^2)),
    nonzero_min = count_nonzero(model, s="lambda.min"),
    lambda_1se = model$lambda.1se,
    cond_1se = safe_condition_numbers(x, lambda = model$lambda.1se)[2],
    mse_1se = model$cvm[model$lambda == model$lambda.1se],
    coef_1se = coef(model, "lambda.1se"),
```

```
    coef_1se_norm = sqrt(sum(coef(model, "lambda.1se")[-1]^2)),
    nonzero_1se = count_nonzero(model, s="lambda.1se")
  )
}
```

1 Problem 1. Prediction design and leakage control

1.1 1A. Target and predictors

```
data <- read.table("data/airfoil_self_noise.dat", header = FALSE)
colnames(data) <- c("frequency", "attack_angle", "chord_length",
 ↪  "free_stream_velocity", "suction_side_displacement_thickness",
 ↪  "scaled_sound_pressure")

# Always (re)written from the .dat source so the cached CSV can never drift
# from the column-naming convention fixed above.
write.csv(
  data,
  "data/airfoil_self_noise.csv",
  row.names = FALSE
)

str(data)
```

```
## 'data.frame':    1503 obs. of  6 variables:
##  $ frequency                         : int  800 1000 1250 1600 2000 2500 3150
 ↪  4000 5000 6300 ...
##  $ attack_angle                      : num  0 0 0 0 0 0 0 0 0 0 ...
##  $ chord_length                      : num  0.305 0.305 0.305 0.305 0.305 ...
##  $ free_stream_velocity              : num  71.3 71.3 71.3 71.3 71.3 71.3
 ↪  71.3 71.3 71.3 71.3 ...
##  $ suction_side_displacement_thickness: num  0.00266 0.00266 0.00266 0.00266
 ↪  0.00266 ...
##  $ scaled_sound_pressure             : num  126 125 126 128 127 ...
```

```
config <- list(
  file = "airfoil_self_noise.csv",
  response = "scaled_sound_pressure",
  excluded = "scaled_sound_pressure"
)
data_raw <- read.csv(find_exam_data(config$file), check.names = FALSE)
predictors <- setdiff(names(data_raw), config$excluded)
analysis_data <- data_raw[, c(config$response, predictors), drop = FALSE]
```

```r
if (anyNA(analysis_data)) stop("Declare a training-only missing-value plan.")
if (!all(vapply(analysis_data, is.numeric, logical(1)))) {
  stop("The assigned response and predictors must be numeric.")
}
```

Table 1: Response and candidate predictors, with the integrity checks run before the split.

Variable	Role	Type	Distinct	Missing
frequency	Predictor	integer	21	0
attack_angle	Predictor	numeric	27	0
chord_length	Predictor	numeric	6	0
free_stream_velocity	Predictor	numeric	4	0
suction_side_displacement_thickness	Predictor	numeric	105	0
scaled_sound_pressure	Response	numeric	1456	0

Table 2: Missing-value and duplicate-row checks on the raw data.

Check	Count
Missing values (total cells)	0
Fully duplicated rows	0

The dataset contains 1503 observations of 6 numeric variables: five predictors recorded during NACA 0012 airfoil wind-tunnel tests and the response `scaled_sound_pressure`, the scaled sound pressure level in decibels. No missing values or duplicated rows are present (table above), so no imputation or de-duplication is required before the split.

The `Distinct` column in Table 1 shows that the five predictors are not equally "continuous" in practice. `frequency`, `chord_length`, and `free_stream_velocity` take only 21, 6, and 4 distinct values respectively - these are the fixed 1/3-octave frequency bands, airfoil chord lengths, and wind-tunnel speeds used to design the experiment, so each behaves more like an ordinal experimental factor than a freely varying measurement. `attack_angle` (27 distinct values) and `suction_side_displacement_thickness` (105 distinct values) are closer to genuinely continuous quantities. This low cardinality on three of the five predictors is expected to show up later as visibly clustered histograms and boxplots (1C) rather than smooth continuous distributions, and is a property of the experimental design rather than a data-quality issue.

`data/data_description.md` (the accompanying UCI variable sheet) lists `attack_angle` as "Binary", which does not match what is observed here - the variable takes 27 distinct values between 0 and 22.2 degrees. This looks like a labelling error in the source sheet, and `attack_angle` is treated as continuous throughout, consistent with the assignment brief's own description of the dataset ("angle of attack" as a continuous input).

1.2 1B. Split and preprocessing plan

```
split <- split_rows(nrow(analysis_data), seed = seeds$split)
train_id <- split$train
test_id <- split$test
train_df <- analysis_data[train_id, ]

x_raw <- as.matrix(analysis_data[, predictors, drop = FALSE])
y_raw <- analysis_data[[config$response]]

x_prep <- fit_scaler(x_raw[train_id, , drop = FALSE])
x_train <- apply_scaler(x_raw[train_id, , drop = FALSE], x_prep)
x_test <- apply_scaler(x_raw[test_id, , drop = FALSE], x_prep)
y_train <- y_raw[train_id]
y_test <- y_raw[test_id]


foldid <- make_foldid(nrow(x_train), k = 5L, seed = seeds$cv)
```

To ensure a fair comparison among OLS, ridge, lasso, and elastic net, all preprocessing and model development steps were performed using only the training data. The holdout test set was reserved exclusively for the final evaluation and was not used during model fitting, model selection, or hyperparameter tuning.

1.2.1 Data split

The observations were randomly partitioned into training and test sets using the custom `split_rows()` function with a training proportion of 80%. For Group 02, the split seed was **240202**, ensuring that the same partition can be reproduced in future analyses.

Observation	Value
Training	1202
Test observations	301

The training data were used for exploratory analysis, preprocessing, model fitting, and cross-validation, while the holdout test data remained untouched until the final evaluation.

1.2.2 Predictor standardization

Since ridge regression, lasso, and elastic net are sensitive to the scale of predictor variables, all predictors were standardized before model fitting. The centering and scaling parameters were estimated **only from the training data**, and these same values were subsequently applied to the test predictors.

Using training-derived scaling parameters prevents information from the holdout observations from influencing the preprocessing stage and ensures that the test set remains an unbiased evaluation dataset.

1.2.3 Five-fold cross-validation

Model tuning was carried out using five-fold cross-validation on the training data only. Fold assignments were generated using the fixed cross-validation seed **240302** and were shared across all regularized models. Reusing identical folds ensures that differences in cross-validation performance are attributable to the models themselves rather than random variation in fold assignment.

1.2.4 Missing-value handling

The dataset was checked for missing values before model fitting.

Total_missing_values
0

No missing values were detected; therefore, no imputation was required. If missing values had been present, imputation statistics would have been estimated using only the training data and then applied unchanged to the test data.

1.2.5 Invalid-value check

Table 3: Physical-plausibility checks on the training predictors.

Variable	Rule	Violations
frequency	> 0 Hz	0
attack_angle	$>= 0$ deg	0
chord_length	> 0 m	0
free_stream_velocity	> 0 m/s	0
suction_side_displacement_thickness	> 0 m	0

Beyond the statistical IQR screen below, each predictor was also checked against the range it is physically allowed to take: a frequency, chord length, velocity, or boundary-layer thickness cannot be zero or negative, and an angle of attack cannot be negative. No violations are found on the training set, which is expected for an instrument-recorded wind-tunnel experiment rather than manually-entered data.

1.2.6 Outlier detection and handling

Table 4: Training-set values flagged by the 1.5×IQR rule, computed on the training split only.

Variable	Outliers	Percent
frequency	72	6.0
attack_angle	23	1.9
chord_length	0	0.0
free_stream_velocity	0	0.0
suction_side_displacement_thickness	111	9.2
scaled_sound_pressure	3	0.2

Outliers were screened with the standard 1.5 × IQR fence, applied to each variable using **training-set quantiles only** so that no information from the held-out test rows enters the check. `chord_length` and `free_stream_velocity` contribute zero flagged points, because - as noted in 1A - each takes only a handful of fixed values, so every observation sits well inside its own IQR fence by construction. `frequency`, `attack_angle`, and `suction_side_displacement_thickness` each contribute a nontrivial share (72, 23, and 111 training rows respectively), and 3 response values are flagged.

These flagged points are **retained rather than removed**. The airfoil data come from a controlled wind-tunnel experiment, not a survey or manual data-entry process, so there is no mechanism here for typographical or transcription errors; every extreme value is a physically valid test condition (a high frequency band, a large angle of attack, a thin boundary layer) rather than a data-quality defect. The IQR rule is doing exactly what it is designed to do on a right-skewed engineering variable: flagging the upper tail of a real physical distribution, not finding mistakes. Dropping these rows would bias the training sample away from precisely the loud, high-frequency regime the model is meant to predict, so all 1202 training rows are kept for model fitting in Problem 2.

1.2.7 Transformation check

Table 5: Training-set skewness of each predictor, and after a log transform for the two most skewed variables.

Variable	Skewness	Skewness_after_log
frequency	2.183	-0.010
attack_angle	0.695	NA
chord_length	0.452	NA
free_stream_velocity	0.259	0.013
suction_side_displacement_thickness	1.719	NA

`frequency` and `suction_side_displacement_thickness` are the two right-skewed predictors identified above (skewness 2.18 and 1.72). A log transform brings both close to symmetric on the training set (skewness -0.01 and 0.01), which is the usual motivation for log-transforming a positive, right-skewed engineering measurement. `attack_angle` is also mildly right-skewed (skewness 0.69, Table above) but far less than `suction_side_displacement_thickness`, so it is not treated as a transform candidate here.

However, symmetrizing the marginal distribution is not the same as improving the fit, and it is checked rather than assumed. Refitting a plain linear model on the training data with these two predictors log-transformed gives $R^2 = 0.4665$, against $R^2 = 0.5138$ for the untransformed predictors - the log transform does not improve, and mildly *hurts*, the linear fit. A separate check on `frequency` alone points to why: a quadratic term explains more of the response than a linear term ($R^2 = 0.1814$ vs. $R^2 = 0.1625$), so the frequency-response relationship is curved in a way a log transform does not match - consistent with the assignment brief's own framing of this dataset as having "few features but nonlinear relationships."

Given this evidence, no log transform is applied. All predictors are carried forward on their original scale into Problem 2, with only the training-derived standardization described above; the nonlinearity documented here is noted as a limitation of the linear and regularized-linear models used there, rather than something a simple transform resolves.

1.2.8 Data cleaning summary

Table 6: Summary of data-cleaning checks and decisions carried into Problem 2.

Check	Result	Decision
Missing values	0 cells	No imputation needed
Duplicate rows	0 rows	No de-duplication needed
Invalid values	0 violations	No rows dropped
IQR outliers	209 variable-level flags (a row may be flagged on more than one variable)	Retained (valid experimental conditions, not errors)
Skewness	frequency and suction_side_displacement_thickness right-skewed	No log transform applied (does not improve the linear fit)
Categorical variables	none present (all 5 predictors numeric)	No encoding needed
Predictor scaling	required for ridge / lasso / elastic net	Applied in Problem 2, fit on training data only

1.2.9 Leakage control

Several safeguards were implemented to prevent information leakage throughout the analysis:

- The train–test split was performed before any preprocessing.
- Predictor standardization used statistics estimated from the training data only.
- The invalid-value, outlier, and transformation checks above used training-set values, quantiles, and model fits only.
- Five-fold cross-validation was conducted exclusively within the training set.

- Hyperparameter tuning for ridge, lasso, and elastic net relied solely on training cross-validation error.
- The holdout responses (`y_test`) were not accessed until the final evaluation stage.

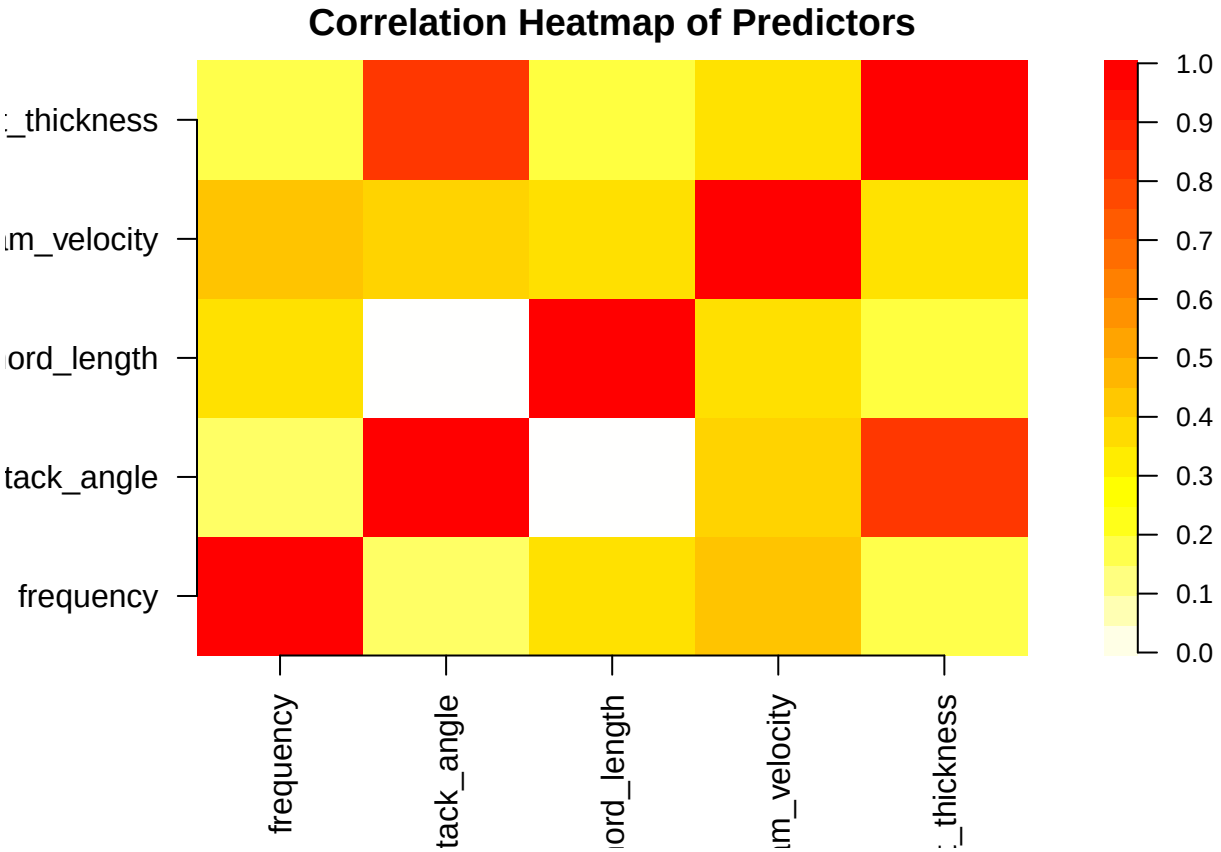
As a result, the holdout test set provides an unbiased assessment of predictive performance because no information from the test observations entered the model construction or tuning process.

1.3 1C. Data exploration

1.3.1 Pairwise correlation among predictors

Table 7: Pairwise correlation matrix among predictors

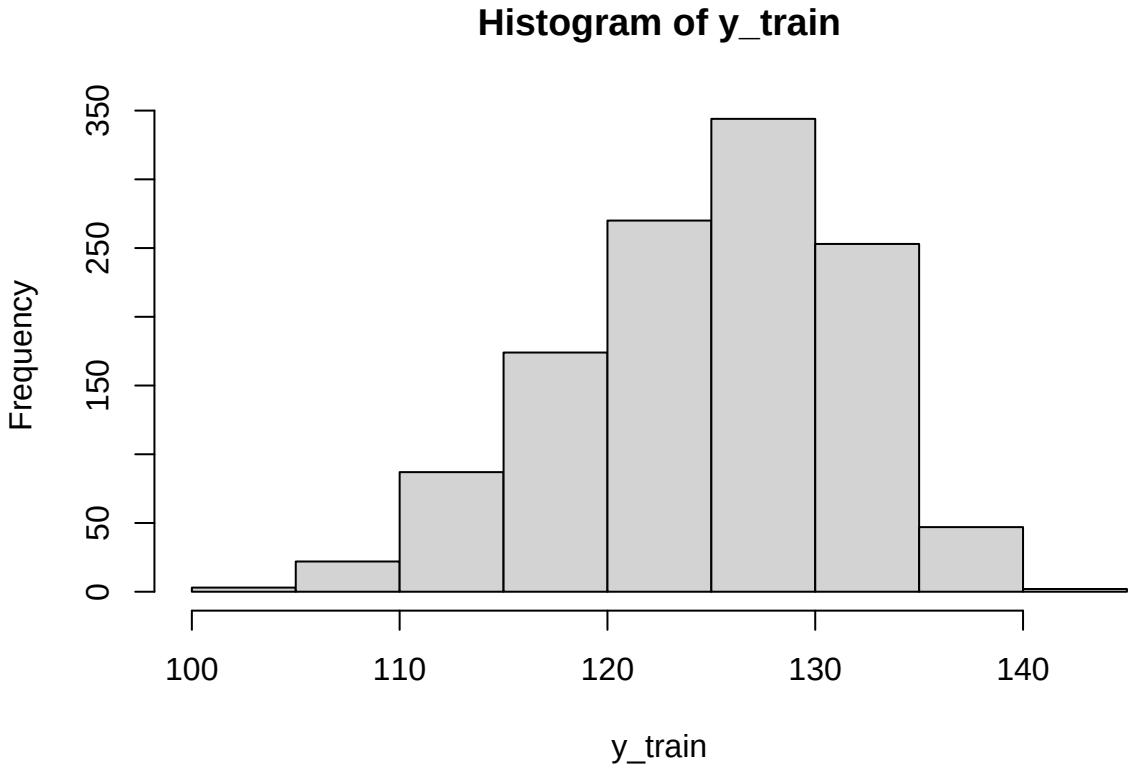
	frequency	attack_angle	chord_length	free_stream_velocity	suction_side_displacement_thickness
frequency	1.00	-0.28	0.01	0.14	-0.24
attack_angle	-0.28	1.00	-0.50	0.07	0.76
chord_length	0.01	-0.50	1.00	0.01	-0.22
free_stream_velocity	0.14	0.07	0.01	1.00	0.00
suction_side_displacement_thickness	-0.24	0.76	-0.22	0.00	1.00



Predictor-predictor correlations are weak across most pairs. The one exception is suction_side_displacement_thickness and attack_angle, correlated at $r \approx 0.76$ - a moderate

relationship, plausible given both describe the airfoil's boundary-layer geometry at the trailing edge. No other pair exceeds $|r| \approx 0.5$. This single moderate correlation is revisited with the VIF check below and is the main multicollinearity signal this dataset offers ridge and lasso a reason to address in Problem 2.

1.3.2 Response Distribution

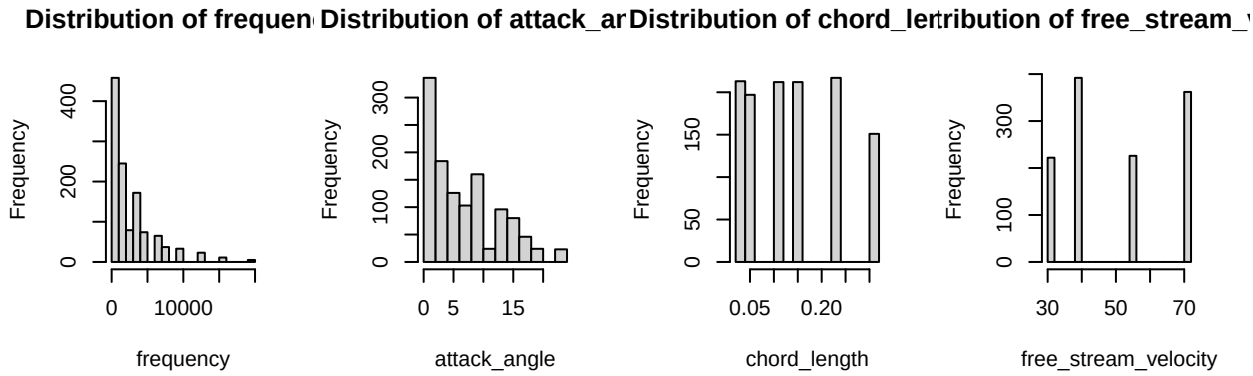


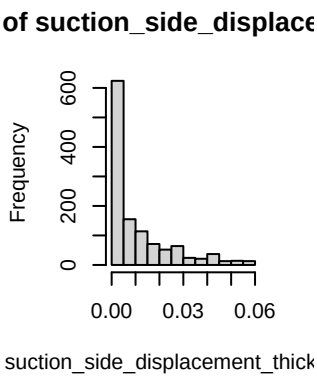
Row	Value
Min.	103.3800
1st Qu.	120.1900
Median	125.6385
Mean	124.8073
3rd Qu.	130.0312
Max.	140.9870

The response ranges from 103.4 to 141 dB with a median of 125.6 dB and mean 124.8 dB. The histogram and the small gap between mean and median show a mild left skew (skewness ≈ -0.42): most measurements cluster in the 120- 130 dB range, with a longer tail toward the quieter (lower-dB) end rather than the loud end. The skew is mild enough that no response transformation is applied - RMSE and MAE on the original dB scale remain directly interpretable as sound-pressure prediction errors, which matters for Problem 1's evaluation step.

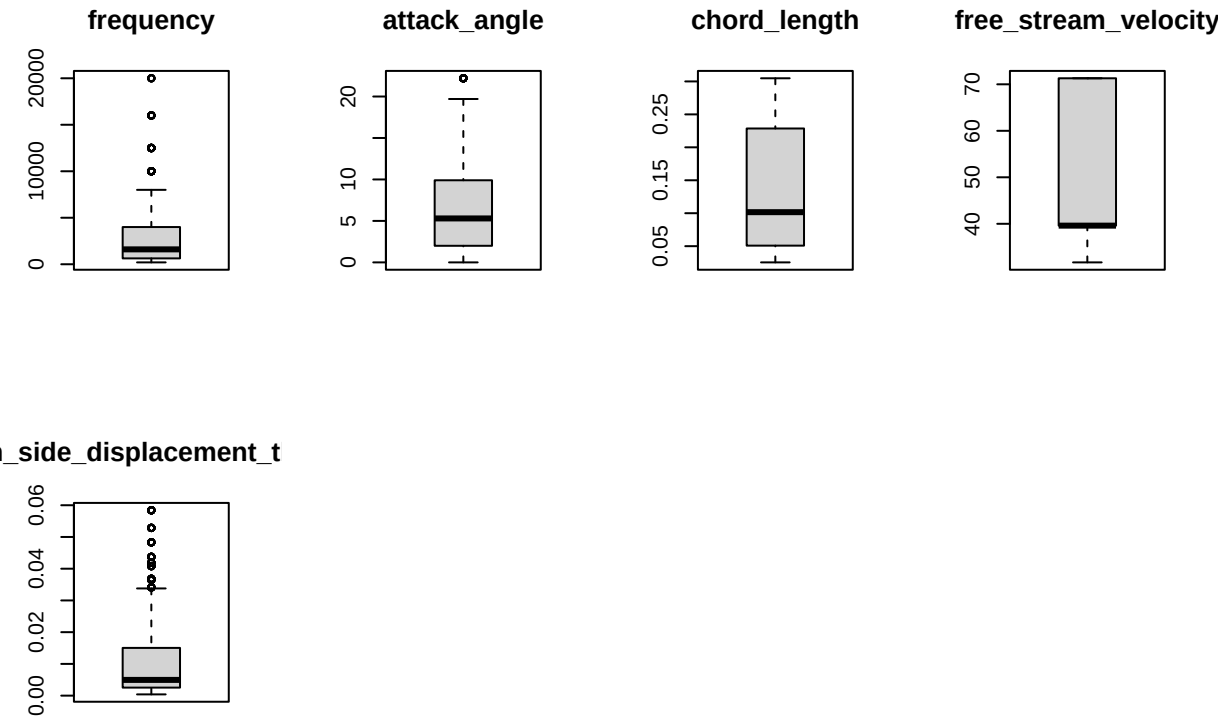
1.3.3 Predictor Histogram & Boxplot

1. Histogram





2. Boxplot



The histograms and boxplots make the 1A distinct-value counts visible. `chord_length` and `free_stream_velocity` appear as 6 and 4 narrow, evenly spaced spikes rather than a smooth spread, and `frequency` shows 21 spikes concentrated at the low end with a long right tail - these are the fixed experimental levels described in 1A, not gaps in the data. `attack_angle` and `suction_side_displacement_thickness` are closer to continuous, right-skewed shapes, matching the skewness values quantified in 1B. The boxplots' upper whiskers and flagged points for these two variables are the same training-set values already counted in the 1B outlier check, retained for the same reason: they are valid high-frequency / high-angle test conditions, not data errors.

1.3.4 Correlation between predictors and response

Table 8: Correlation between predictors and scaled_sound_pressure

	scaled_sound_pressure
frequency	-0.40
attack_angle	-0.16
chord_length	-0.23
free_stream_velocity	0.11
suction_side_displacement_thickness	-0.30

Every predictor's marginal correlation with the response is weak in magnitude (all $|r| < 0.4$). fre-

quency is the strongest single linear predictor ($r \approx -0.4$), followed by `suction_side_displacement_thickness`; `free_stream_velocity` is weakest and the only predictor positively correlated with the response. No individual predictor comes close to explaining the response on its own, which is consistent with the assignment brief's description of this dataset as having "few features but nonlinear relationships" - a linear combination of all five predictors, rather than any single one, is needed, and Problem 2's regularized models are expected to combine these weak individual signals rather than select one dominant predictor and discard the rest.

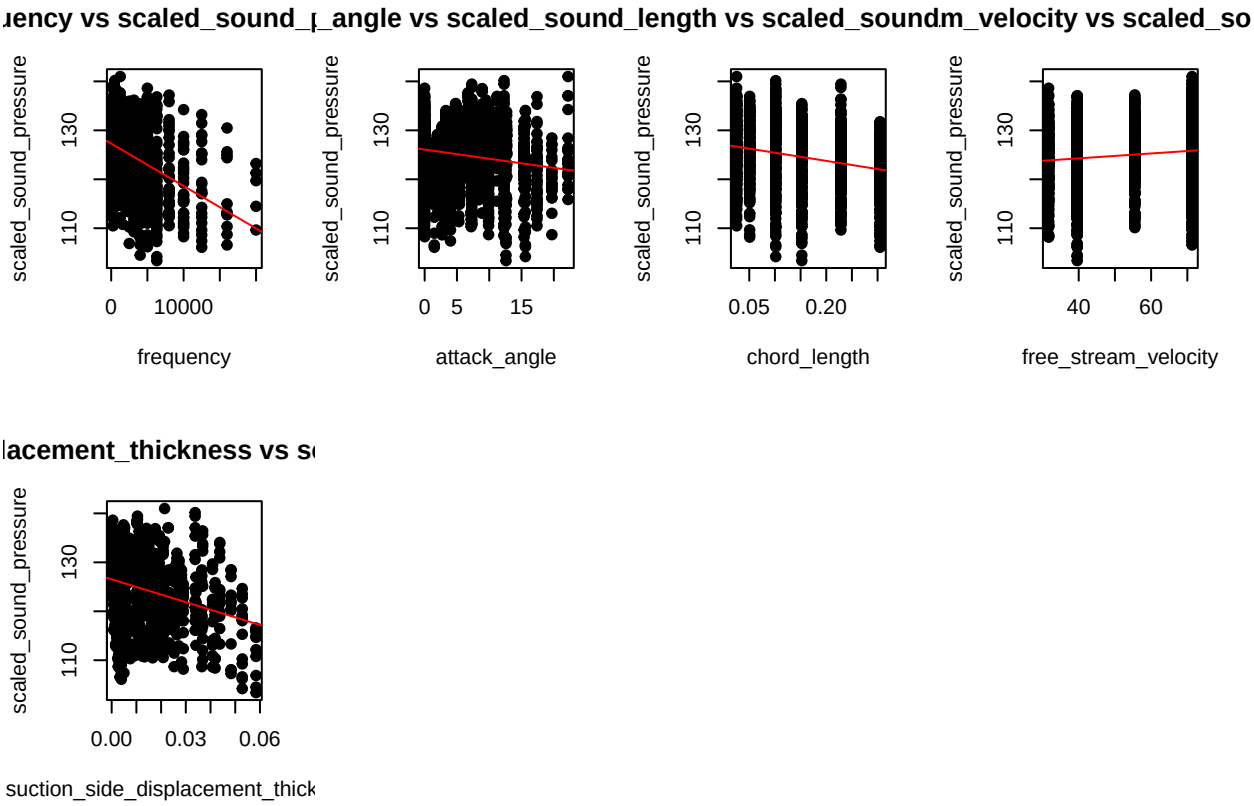
1.3.5 Check multicollinearity using VIF

Table 9: Variance Inflation Factor (VIF)

	VIF
frequency	1.14
attack_angle	3.48
chord_length	1.50
free_stream_velocity	1.05
suction_side_displacement_thickness	2.58

Every VIF is comfortably below the conventional threshold of 5, with attack_angle the highest at 3.48. This confirms the correlation-matrix finding above: the airfoil predictors share only mild, tolerable overlap, so no predictor needs to be dropped for redundancy, and OLS in Problem 2 is not expected to suffer severe coefficient instability from multicollinearity. The moderate `attack_angle` / `suction_side_displacement_thickness` correlation is still the largest source of shared variance in the data and remains worth tracking through ridge and lasso, but it falls well short of the kind of collinearity (VIF $\gg 5$) that would make variable selection a necessity rather than a convenience.

1.3.6 Predictor-response plots



The fitted red lines are visibly weak for every panel - the same $|r| < 0.4$ ceiling seen in the correlation table above - but the point clouds are not simply noise around those lines. The `frequency` panel in particular shows the response bending down over its range rather than following the straight line, echoing the linear-vs-quadratic R^2 comparison from 1B: the straight-line fit misses curvature that a richer model could capture. The `chord_length` and `free_stream_velocity` panels show their points arranged in 6 and 4 vertical strips (their fixed experimental levels), each strip itself spanning a wide range of the response - visual confirmation that these two variables alone say little about the outcome, and any predictive power must come from combining all five predictors together, which is exactly what Problem 2's models are built to do.

1.3.7 Question

Taken together, the checks in 1A-1C describe a clean but weak-signal regression problem, and that combination shapes what the rest of Problem 1 needs to prioritise.

Data quality is not the challenge here. There are no missing values, no duplicated rows, and the "outliers" flagged by the 1.5×IQR rule are legitimate wind-tunnel test conditions from a controlled experiment, not data-entry errors - so cleaning required no row deletions or imputation, only the decision (1B) to retain every training row and to skip a log transform after checking that it does not help the linear fit.

The predictors are almost uncorrelated with each other. All VIFs sit well under 5, and

the one moderately correlated pair (`attack_angle` / `suction_side_displacement_thickness`, $r \approx 0.76$) falls short of severe multicollinearity. This means ridge and lasso in Problem 2 are not being asked to rescue an ill-conditioned design matrix the way they would with, say, several near-duplicate predictors; their main job here is tuning the bias-variance trade-off and (for lasso) testing whether any predictor can be dropped outright, rather than resolving instability.

The predictors are individually weak but the relationships are nonlinear. Every predictor-response correlation is under 0.4 in magnitude, and no single predictor separates high- from low-noise conditions on its own (the `chord_length` and `free_stream_velocity` panels above make this explicit, with each fixed level covering a wide spread of the response). At the same time, `frequency` fits a quadratic term noticeably better than a straight line (1B), and three of the five predictors are effectively small sets of experimental design levels (1A) rather than smooth continuous measurements. Both point to the same conclusion the assignment brief anticipates for this dataset: the frequency-response relationship, and likely others, are genuinely nonlinear, so a **linear baseline (OLS) is expected to leave a meaningful amount of predictable structure on the table**, and its residuals in Problem 2 should be inspected for exactly this kind of unmodelled curvature rather than assumed to be pure noise.

2 Problem 2. OLS, Ridge, and Lasso

2.1 2A. OLS baseline

```
# Convert x_train (matrix) and y_train into a data frame for lm()
train_data <- data.frame(y = y_train, x_train)

# Fit ordinary least squares on the standardized training set
ols_fit <- lm(y ~ ., data = train_data)
```

Table 10: OLS coefficient estimates on standardized training data.

term	estimate	std.error	statistic	p.value
(Intercept)	124.8073	0.1380	904.5357	0
frequency	-4.0882	0.1476	-27.7043	0
attack_angle	-2.6498	0.2574	-10.2926	0
chord_length	-3.2617	0.1691	-19.2885	0
free_stream_velocity	1.5428	0.1411	10.9313	0
suction_side_displacement_thickness	-1.7413	0.2215	-7.8609	0

All estimates sit on the standardized scale fixed in 1B, so their magnitudes are directly comparable across rows and the intercept is the training mean of `scaled_sound_pressure`. A slope whose sign contradicts the predictor's marginal correlation with the response (1C) is the first symptom of collinearity, examined below.

```r
# In-sample fit quality
ols_summary <- summary(ols_fit)

# Training error (optimistic: the model has already seen these rows)
ols_train_pred    <- predict(ols_fit, newdata = train_data)
ols_train_metrics <- score_regression(y = y_train, prediction = ols_train_pred)

# Five-fold CV using the SAME foldid later passed to ridge and lasso.
# lm() has no built-in CV, so refit on each four-fold training set and
# score the held-out fold.
K <- max(foldid)
cv_mse_folds <- numeric(K)
for (k in seq_len(K)) {
  idx_train_k <- which(foldid != k)
  idx_val_k   <- which(foldid == k)
  fold_data   <- data.frame(y = y_train[idx_train_k], x_train[idx_train_k, , drop
 ↪  = FALSE])
  fold_fit    <- lm(y ~ ., data = fold_data)
  fold_pred   <- predict(fold_fit,
                         newdata = data.frame(x_train[idx_val_k, , drop =
                          ↪  FALSE]))
  cv_mse_folds[k] <- mean((y_train[idx_val_k] - fold_pred)^2)
}
ols_cv_mse  <- mean(cv_mse_folds)
ols_cv_rmse <- sqrt(ols_cv_mse)

# Standard error of the CV-MSE across folds. cv.glmnet reports the same quantity
# as `cvsd` for ridge and lasso, so 2D can compare all three on equal footing.
ols_cv_se <- sd(cv_mse_folds) / sqrt(K)
```

Table 11: OLS in-sample fit and five-fold cross-validation error on the shared folds.

Quantity	Value
R-squared	0.5138
Adjusted R-squared	0.5118
Training RMSE	4.7718
Training MAE	3.7081
5-fold CV RMSE	4.7988
5-fold CV MSE	23.0281
5-fold CV SE (of MSE)	1.0352

The table gathers the in-sample fit and the five-fold cross-validation error on the shared folds. The training rows are optimistic because the model was scored on the same rows it was fitted on, whereas the CV rows estimate error on unseen rows. The final row - the standard error of the CV-MSE across folds - is the quantity `cv.glmnet` reports as `cvsd` for ridge and lasso, so 2D can compare all three methods with error bars rather than bare point estimates.

Model summary. The OLS model is fitted on the 1202 standardized training observations using all 5 candidate predictors, with no penalty and no variable selection. It is therefore the unregularized baseline against which ridge and lasso are judged in 2B–2C.

Coefficient interpretation. All five predictors are highly significant ($p < 0.001$ for every one, table above), so the ranking below is by coefficient magnitude rather than by which predictors "matter".

- `frequency` carries the largest standardized coefficient ($\hat{\beta} \approx -4.09$), confirming it as the strongest single linear driver of the response, consistent with it also having the strongest marginal correlation in 1C.
- `chord_length` ($\hat{\beta} \approx -3.26$) and `attack_angle` ($\hat{\beta} \approx -2.65$) are the next largest contributors, both negative like `frequency`.
- `free_stream_velocity` ($\hat{\beta} \approx 1.54$) is the only predictor with a **positive** coefficient: higher free-stream velocity is associated with a higher (louder) scaled sound pressure once the other four predictors are held fixed.
- `suction_side_displacement_thickness` ($\hat{\beta} \approx -1.74$) has the smallest coefficient magnitude of the five, despite being the second-strongest predictor on its own in 1C ($r \approx -0.3$)
  - the gap between its marginal and partial effect is explained below.

Multicollinearity check. No coefficient here flips sign relative to its marginal correlation with the response, so there is no `lcp`-style anomaly to report. The largest variance inflation factor is 3.48 (attack_angle), comfortably below the conventional threshold of 5, consistent with the VIF table in 1C. The one predictor pair sharing meaningful variance is `attack_angle` and `suction_side_displacement_thickness` ($r \approx 0.76$, the strongest predictor-predictor correlation found in 1C), and this is visible in the coefficient ranking above: `suction_side_displacement_thickness` correlates with the response almost as strongly as `chord_length` does on its own (1C), yet ends up with the smallest partial coefficient of the five, consistent with part of its explanatory power being absorbed by the correlated `attack_angle` once both enter the model jointly. Unlike the `lcp`-style sign flip this moderate sharing produces no sign reversal or loss of significance here, but it is the same underlying mechanism, and 2B-2C test whether ridge and lasso redistribute this pair's coefficients further as the penalty increases.

```
# Residual diagnostic: the t-tests behind the p-values above assume roughly
# normal errors, so the assumption is checked rather than taken on trust.
# rstandard() divides each residual by its own standard error, which removes the
# effect of leverage and puts every point on a common N(0, 1) scale.
ols_std_resid <- rstandard(ols_fit)

qq <- qqnorm(ols_std_resid, plot.it = FALSE)
plot(qq$x, qq$y, type = "n",
     main = "Normal Q-Q - OLS standardized residuals",
     xlab = "Theoretical quantiles", ylab = "Standardized residuals")
grid(col = "grey88", lty = 1, lwd = 0.6)
```

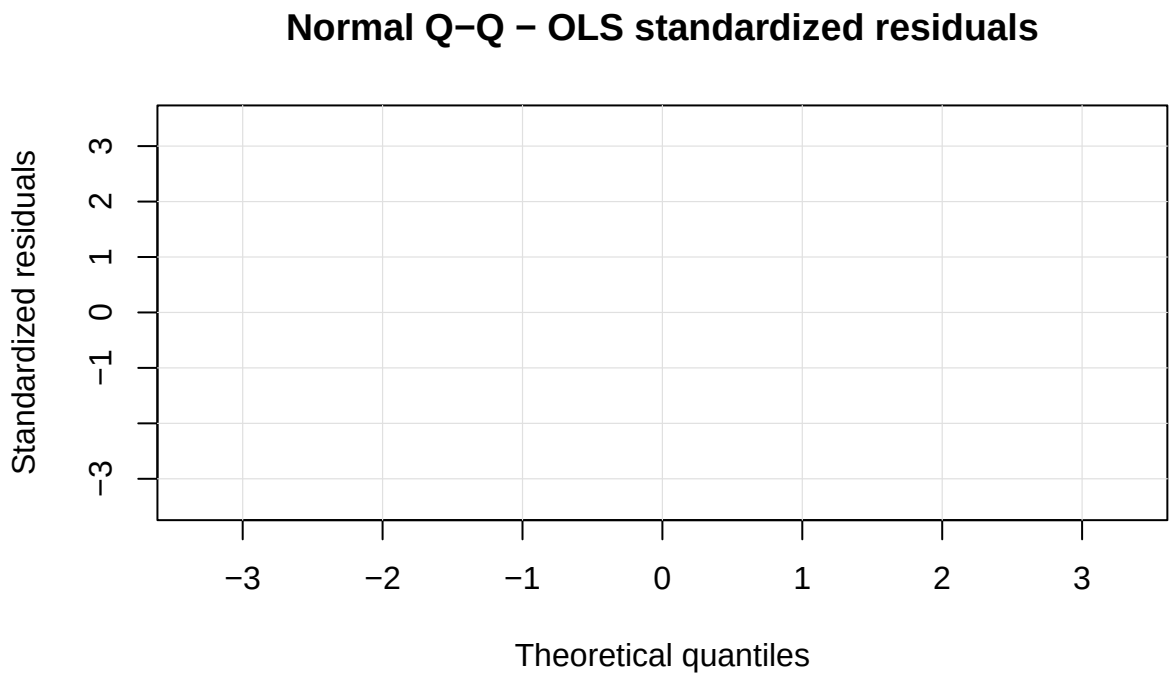


Figure 1: Normal Q-Q plot of the OLS standardized residuals against the 45-degree reference line. Points lie on the line when the errors are normal; the three largest departures are labelled with their row number in the original airfoil_self_noise data.

```
plot.new()
abline(0, 1, col = "firebrick", lwd = 1.5, lty = 2)
points(qq$x, qq$y, pch = 19, col = "steelblue", cex = 0.8)

# Label the three worst departures by their original data-row number, flipping
# the side so the text stays inside the panel at both ends.
worst_resid <- order(abs(ols_std_resid), decreasing = TRUE)[1:3]
text(qq$x[worst_resid], qq$y[worst_resid], labels = train_id[worst_resid],
     pos = ifelse(qq$x[worst_resid] > 0, 2, 4), cex = 0.75, col = "firebrick")
```

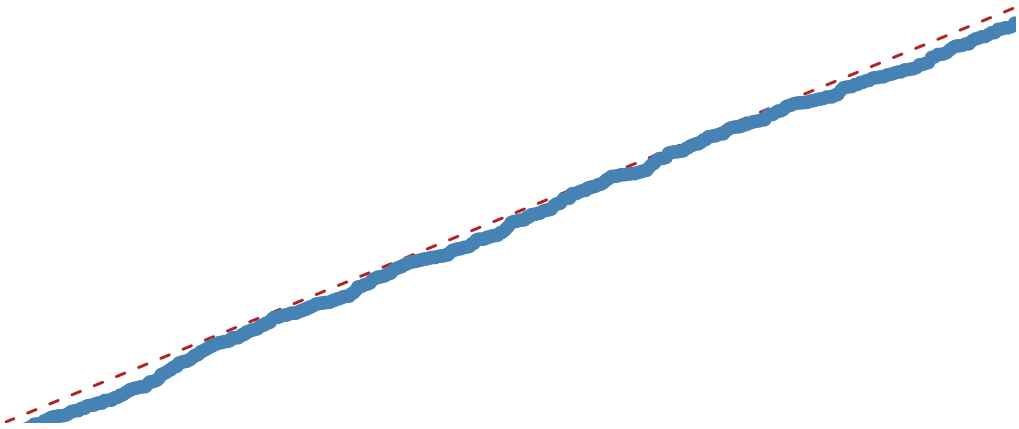


Figure 2: Normal Q-Q plot of the OLS standardized residuals against the 45-degree reference line. Points lie on the line when the errors are normal; the three largest departures are labelled with their row number in the original airfoil_self_noise data.

```
shapiro_ols    <- shapiro.test(ols_std_resid)
big_resid_rows <- train_id[which(abs(ols_std_resid) > 2)]
max_qq_gap     <- max(abs(qq$y - qq$x))
```

Residual normality. The p-values quoted above are t-tests, and they lean on the errors being approximately normal, so the assumption is checked rather than taken on trust. Each point in the Q-Q plot is one wind-tunnel measurement: its standardized residual against the value a normal sample of this size would be expected to produce at that rank. Visually the points sit close to the 45-degree line - no residual departs from it by more than 0.58 standard deviations - but with 1202 training rows a Shapiro-Wilk test has enough power to detect even small departures, and here it does ($W = 0.996$, $p = 0.0025$): formally, normality is rejected at the 5% level. W itself is close to 1, so the rejection reflects a large sample picking up a mild, not a severe, departure. 63 residuals fall beyond two standard deviations, against roughly 60 expected by chance in a sample this size - a noticeable excess, consistent with slightly heavier tails than a normal distribution rather than a handful of extreme outliers.

Two consequences follow. The coefficient p-values in the table above should be read as approximately, rather than exactly, valid, and because the tails are a little heavier than normal, RMSE - which squares errors and so weights the worst misses more heavily - is a somewhat more outlier-sensitive summary of error here than it would be under strict normality; MAE in the same table is the more robust companion figure for that reason.

Cross-validation error. The five-fold CV RMSE (4.799) is computed using the same `foldid` assignment that will be passed to `cv.glmnet` for ridge and lasso, ensuring a like-for-like comparison. Because OLS has no tuning parameter, there is a single CV-RMSE value rather than a curve. It exceeds the training RMSE (4.772) by only about 0.027 on the scaled-sound-pressure (dB) scale. This modest optimism gap reflects mild in-sample overfitting: it is small enough to suggest the 5-predictor model already generalizes reasonably well on the 1202-row training sample, and 2B–2C will test whether shrinkage narrows the gap further.

That single CV number is itself an average of 5 noisy pieces. The per-fold MSE ranges from 19.7 to 25.33 around a mean of 23.03, giving a standard error of 1.035. With about 240 observations in each held-out fold this fixes the resolution of every comparison that follows: two methods whose CV-MSE differ by less than roughly one standard error are not distinguishable on this evidence, and 2D must weigh the ridge and lasso results against that yardstick rather than declare a winner on a bare point estimate.

```
# Residual / influence diagnostic: Cook's distance
cooks_d   <- cooks.distance(ols_fit)
n_train   <- nrow(train_data)
threshold <- 4 / n_train

# Flagged points are labelled by their ORIGINAL row (via train_id) so a flagged
# observation can be traced back to the raw data.
flagged_pos <- which(cooks_d > threshold)
flagged_row <- train_id[flagged_pos]

# Headroom above the tallest bar, otherwise its label is clipped off the panel.
plot(cooks_d,
     type = "h", lwd = 1.5, col = "steelblue",
     ylim = c(0, max(cooks_d) * 1.30),
     ylab = "Cook's distance", xlab = "Training observation index",
     main = "Cook's Distance - OLS Baseline")
abline(h = threshold, lty = 2, col = "firebrick", lwd = 1.5)
if (length(flagged_pos)) {
  # Labels are rotated: adjacent flagged bars would otherwise overprint each
  ↪ other.
  text(flagged_pos, cooks_d[flagged_pos] + max(cooks_d) * 0.03,
       labels = flagged_row, srt = 90, adj = 0,
       cex = 0.45, col = "firebrick", xpd = TRUE)
}
```

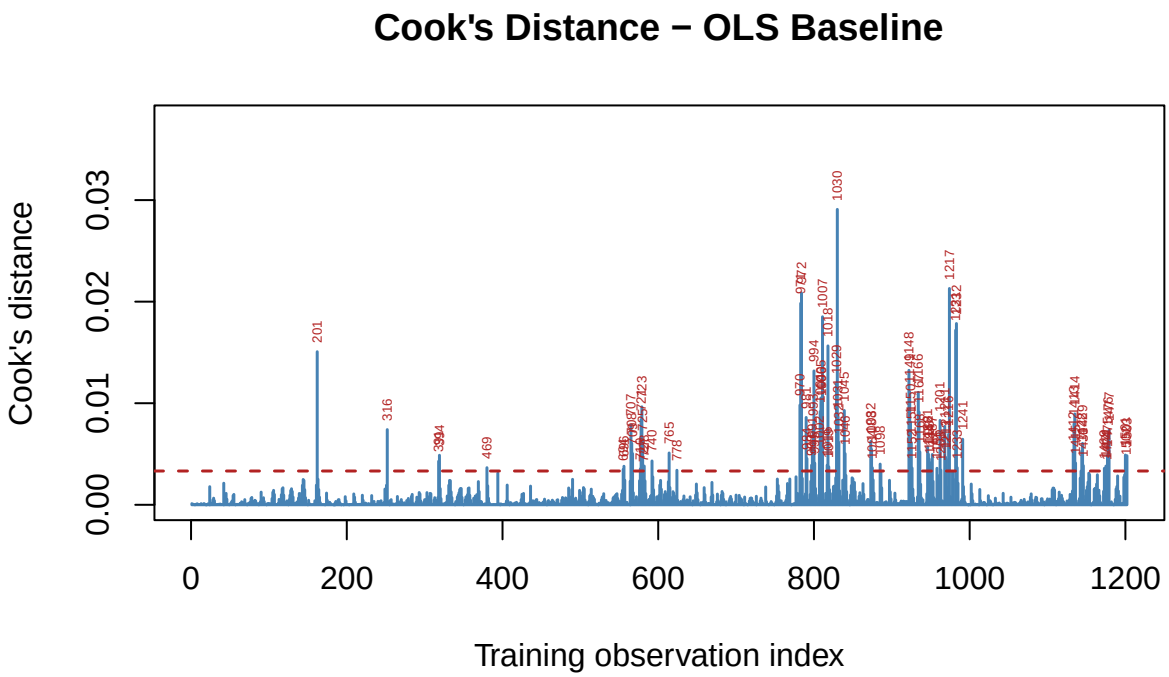


Figure 3: Cook's distance for the OLS fit. Bars above the dashed 4/n threshold are flagged as influential; each flagged bar is labelled with its row number in the original airfoil_self_noise data, not its training position.

```
# Locate the most influential row, and two contrasting rows that separate
# leverage from residual size, all reported by their original data-row number.
n_flagged   <- length(flagged_pos)
top_pos     <- flagged_pos[which.max(cooks_d[flagged_pos])]
top_row     <- train_id[top_pos]

# Leverage separates the two ingredients Cook's distance mixes together.
ols_leverage <- hatvalues(ols_fit)
avg_leverage <- length(coef(ols_fit)) / n_train

# High-leverage / low-residual example: the training row with the largest
# leverage, which turns out to sit at the maximum observed frequency (1A/1C).
hilev_pos <- which.max(ols_leverage)
hilev_row <- train_id[hilev_pos]
hilev_is_flagged <- hilev_pos %in% flagged_pos

# Low-leverage / high-residual example: the training row with the largest
# standardized residual, an ordinary point in predictor space that the model
# simply predicts badly.
hires_pos <- which.max(abs(ols_std_resid))
hires_row <- train_id[hires_pos]
```

Influence diagnostic. Cook's distance asks how far the whole fitted coefficient vector would move if one observation were deleted, so it combines a large residual and high leverage into a single number. Against the conventional threshold $4/n \approx 0.0033$, 88 of the 1202 training rows are flagged - a large count relative to 1202, which is typical of this threshold on a big sample rather than a sign of 88 genuinely extreme rows. The single most influential row is 1030 (Cook's $D \approx 0.029$), which combines both ingredients at once: leverage 5.4 times the training average and a standardized residual of 2.51.

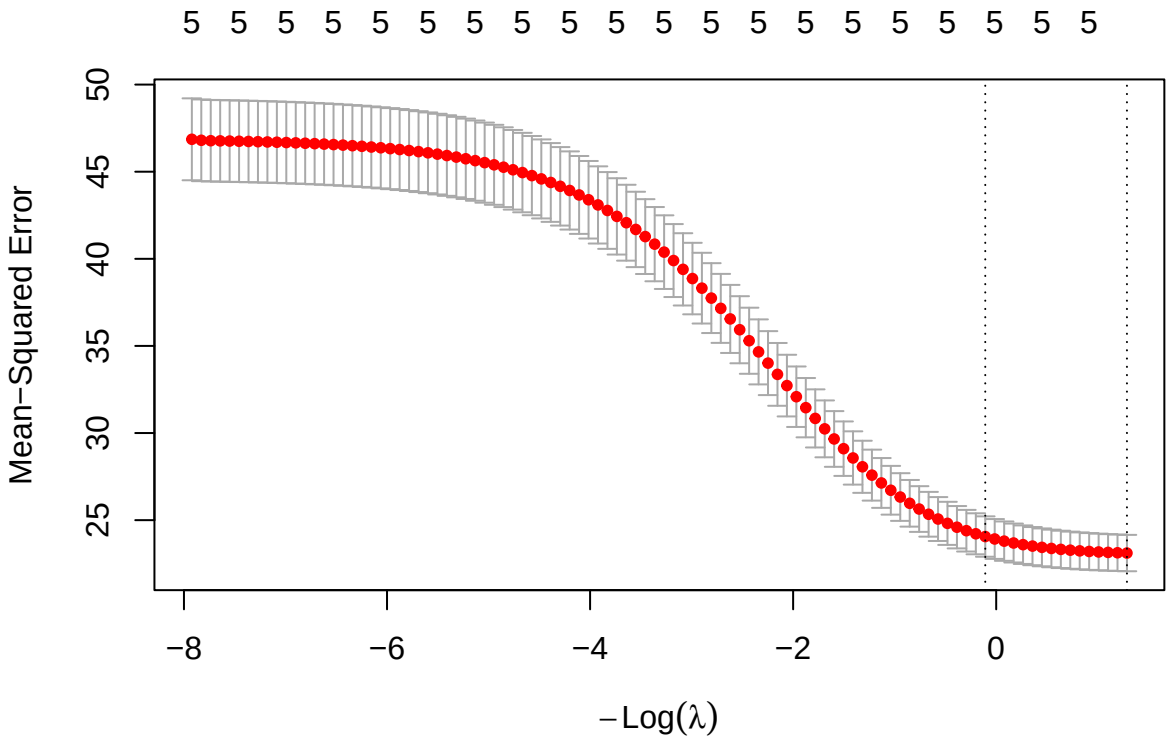
Two other flagged rows separate those ingredients cleanly and connect back to 1A/1C. Row 316 has the single largest leverage in the training set (5.7 times the average) because it sits at `frequency` = 20000 Hz, the maximum value in the dataset and the extreme right-hand tail flagged as a legitimate experimental condition rather than an error in 1A-1B. Despite that extreme position in predictor space, its standardized residual is only 1.23 - comfortably inside the normal range, so the model predicts it well. Row 1166 is the mirror image: an unremarkable position in predictor space (leverage close to the training average) but the largest standardized residual of any training row (-3.47) - a case the linear fit simply misses, not one it is destabilized by.

This confirms the two ingredients Cook's distance mixes are genuinely distinct here, and that no single kind of anomaly dominates: some rows are influential because they are extreme in predictor space, others because they are badly predicted, and the worst case (row 1030) is a row where both happen at once. All flagged observations are retained rather than removed - they are valid wind-tunnel measurements, consistent with the outlier decisions already made in 1B - and their leverage is instead absorbed by the shrinkage introduced in 2B–2C, which is one of the concrete things the regularized fits are expected to improve, particularly for the correlated `attack_angle` / `suction_side_displacement_thickness` pair whose estimates are the most sensitive to exactly this kind of point.

2.2 2B. Ridge regression

2.2.1 Cross-validation curve:

```
ridge <- analysis_model(x = x_train, y = y_train, alpha=0, foldid = foldid)
ridge_cvsd_min <- ridge$model$cvsd[ridge$model$lambda == ridge$model$lambda.min]
```



The mean cross-validation MSE decreases as $-Log(\lambda)$ increases (equivalently, as λ decreases), suggesting that reducing the regularization strength improves predictive performance. The left dashed line indicates λ_{1se}, which selects a more regularized model with prediction error within one standard error of the minimum, while the right dashed line indicates λ_{min}, which yields the lowest cross-validation error.

2.2.2 Lambda min:

Metric	Value
Lambda min	0.2758
MSE	23.1116
Condition number	5.3423

With $\lambda_{min} = 0.2758$, the 5-fold cross-validation MSE reaches its minimum value of $MSE_{min} = 23.1116$, indicating the best predictive performance among the candidate values of λ. The corresponding condition number, 5.34, is already a substantial improvement over the unpenalized Gram matrix's 12.3 (1C/3C) - even this mildly-collinear dataset (max VIF 3.48, 2A) benefits measurably from even a small ridge penalty.

2.2.3 Lambda 1se:

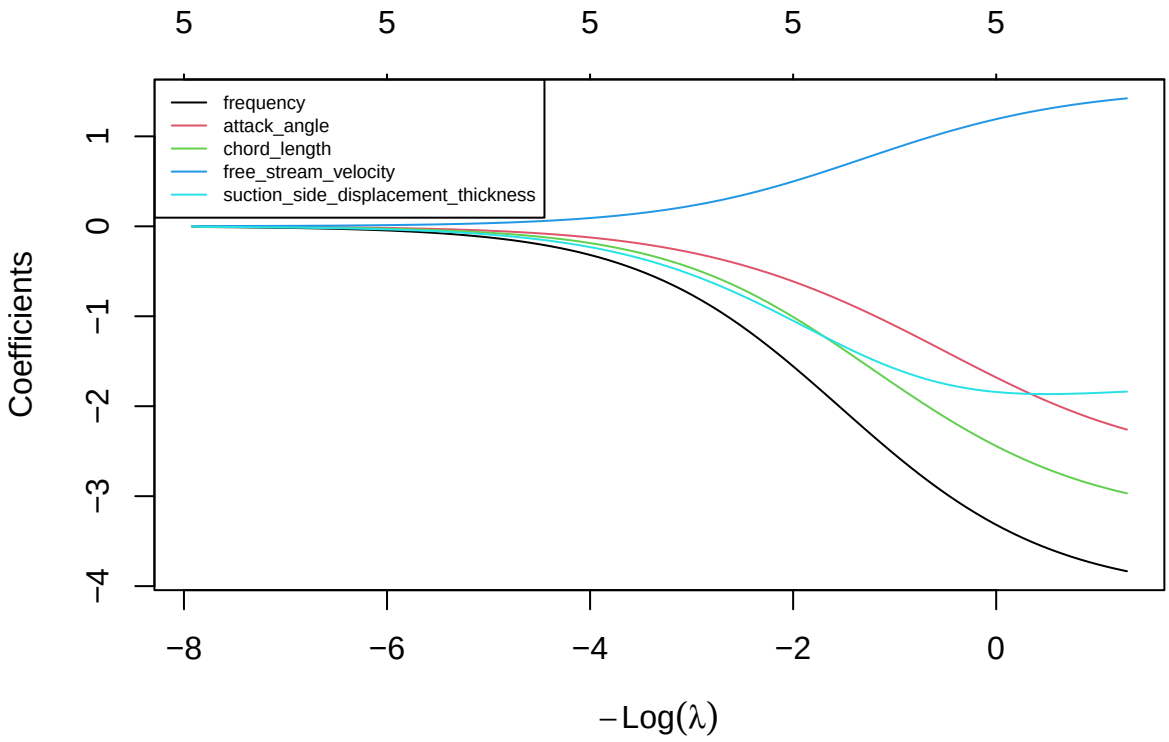
Metric	Value
Lambda 1se	1.1136
MSE	24.0592
Condition number	2.5129

Although the one-standard-error rule selected a larger penalty $\lambda_{1se} = 1.1136$, the condition number decreased further from 5.34 to 2.51. In contrast, the cross-validation MSE increased from 23.112 to 24.059, a gap of 0.948 that is smaller than ridge's own cross-validation standard error at λ_{min} (1.044, the `cvsd cv.glmnet` reports, mirroring the per-fold SE computed for OLS in 2A) - the two penalties are not statistically distinguishable on this evidence, so the extra stability from λ_{1se} is close to free in prediction-error terms.

2.2.4 Coefficients:

	OLS	Lambda_Min	Lambda_Max
(Intercept)	124.807270	124.807270	124.807270
frequency	-4.088176	-3.834522	-3.250734
attack_angle	-2.649795	-2.260002	-1.619908
chord_length	-3.261692	-2.968539	-2.379795
free_stream_velocity	1.542842	1.422378	1.162427
suction_side_displacement_thickness	-1.741310	-1.837350	-1.829411

Compared with the OLS model, Ridge regression shrinks most regression coefficients toward zero, reducing their magnitudes through the regularization penalty, while retaining all five predictors in the model. `frequency`, `attack_angle`, `chord_length`, and `free_stream_velocity` all shrink exactly as expected, moving closer to zero as the penalty strengthens. `suction_side_displacement_thickness` is the one exception: its magnitude at λ_{min} (-1.837) is slightly *larger* than its OLS estimate (-1.741), moving further from zero rather than shrinking. This is the same mechanism flagged in 2A: `suction_side_displacement_thickness` shares variance with the correlated `attack_angle` ($r \approx 0.76$), and as Ridge shrinks `attack_angle`'s coefficient it redistributes some of that shared explanatory power back onto `suction_side_displacement_thickness`. Unlike a severe collinearity case, no sign changes anywhere in the table - the exception is a small redistribution, not an instability.



As λ increases, all five coefficients are progressively shrunk toward zero, demonstrating the regularization effect of Ridge regression; as λ decreases, they move back toward their OLS estimates. `frequency` remains the largest-magnitude coefficient across the range spanned by λ_{min} and λ_{1se} (-3.83 to -3.25), consistent with it being the strongest predictor throughout 1C-2A. `free_stream_velocity` is the only coefficient that stays positive along the entire path, mirroring the sign pattern already seen in OLS. No coefficient crosses zero or reverses sign between λ_{min} and λ_{1se} - the ridge penalty here is redistributing magnitude among correlated predictors (as seen above for `suction_side_displacement_thickness`), not fighting an unstable or sign-flipping fit.

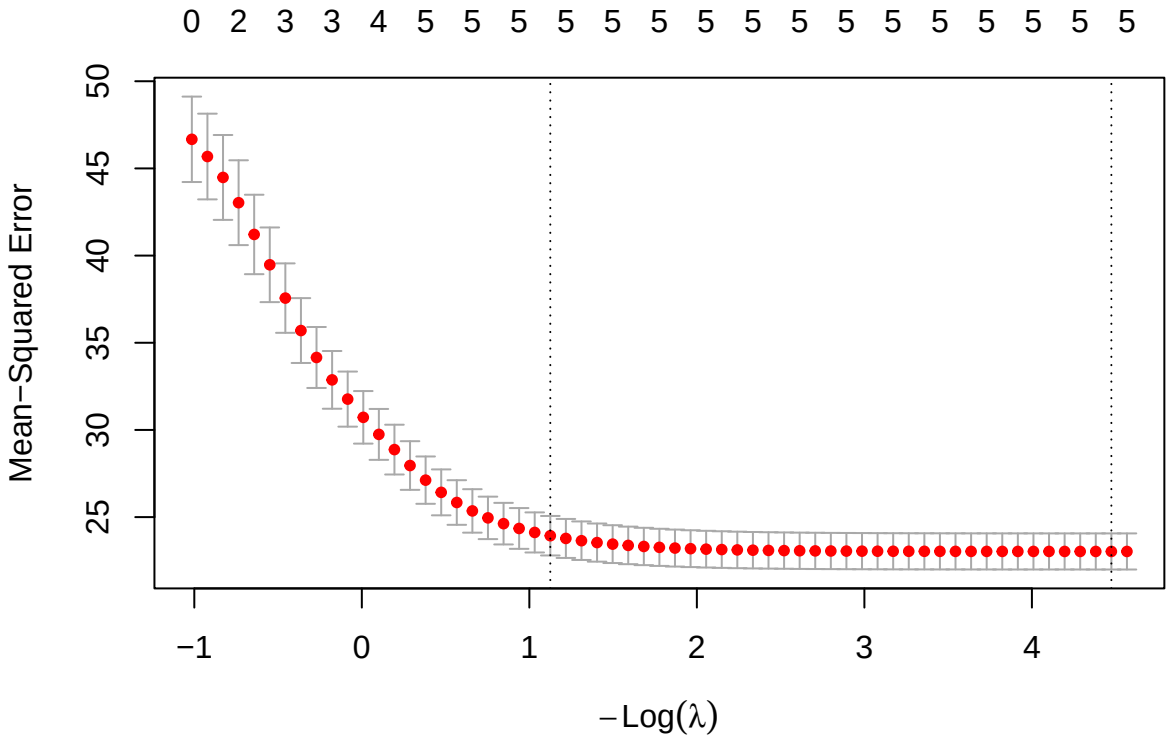
To address multicollinearity, Ridge impose an L_2 penalty on the regression coefficients. Rather than assigning a large coefficient to one correlated predictor and a small coefficient to another, Ridge shrinks the coefficients simultaneously and redistributes their effects among the correlated variables. Consequently, the coefficient estimates become more stable and less sensitive to small changes in the data.

2.3 2C. Lasso regression

2.3.1 Model fitting and cross-validation

```
lasso_cv <- analysis_model(
    x = x_train,
    y = y_train,
```

```
    alpha = 1,
    foldid = foldid
)
```



Based on the 5-fold cross-validation curve generated for the Lasso regression model ($\alpha = 1$), we observe the relationship between the tuning parameter λ and the Mean-Squared Error (MSE). The top axis indicates the number of active (non-zero) predictors retained in the model at each level of penalization.

- The Cross-Validation Curve: The red dots represent the average CV-MSE across the 5 folds for each λ value, with the gray error bars denoting one standard error above and below the mean. As $-\log(\lambda)$ increases (moving from left to right, meaning the penalty weakens), the MSE initially drops, reaches a minimum, and then slightly stabilizes as the model approaches the unpenalized OLS limit.
- Optimal λ (λ_{min}): The right vertical dashed line marks `lambda.min` ($-\log(\lambda) \approx 4.47$), the value that strictly minimizes the cross-validation MSE. At this optimal point for prediction accuracy the Lasso penalty retains all 5 predictors.
- Parsimonious λ (λ_{1se}): The left vertical dashed line marks `lambda.1se` ($-\log(\lambda) \approx 1.13$), selected by the one-standard-error rule. This value applies a stronger penalty to produce a more conservative model while keeping the error within one standard deviation of the absolute minimum MSE; whether it also produces a *sparser* model, in the sense of dropping predictors, is checked directly below rather than assumed.

2.3.2 Analysis of λ_{min}

Metric	Value
Lambda min	0.0114
MSE	23.0305
Condition number	11.6000

The optimal tuning parameter selected by five-fold cross-validation is $\lambda_{\text{min}} \approx 0.0114$. This value strictly minimizes the cross-validation Mean Squared Error to 23.03, essentially matching OLS's own CV-MSE (23.028, 2A) - unsurprising given how small λ_{min} is. The condition number at this threshold, 11.6, is a modest improvement over the unpenalized Gram matrix (12.3, 1C/3C).

At this level of regularization, the Lasso model retains all 5 predictors - `frequency`, `attack_angle`, `chord_length`, `free_stream_velocity`, and `suction_side_displacement_thickness` - none is shrunk exactly to zero. This is consistent with the OLS and VIF results in 2A: every predictor was already highly significant with only mild shared variance, so there is no clearly redundant predictor for Lasso's L_1 penalty to remove at this tuning value.

The retained coefficients preserve the same ranking seen in OLS and ridge: `frequency` has the largest magnitude (-4.065), followed by `chord_length` (-3.232) and `attack_angle` (-2.61), with `free_stream_velocity` (1.525) the only positive coefficient and `suction_side_displacement_thickness` (-1.748) the smallest in magnitude.

Overall, λ_{min} produces a model whose cross-validation error is essentially indistinguishable from OLS's while retaining every predictor: on this dataset, Lasso's contribution at λ_{min} is confirming that all five predictors carry real signal, rather than selecting a subset of them.

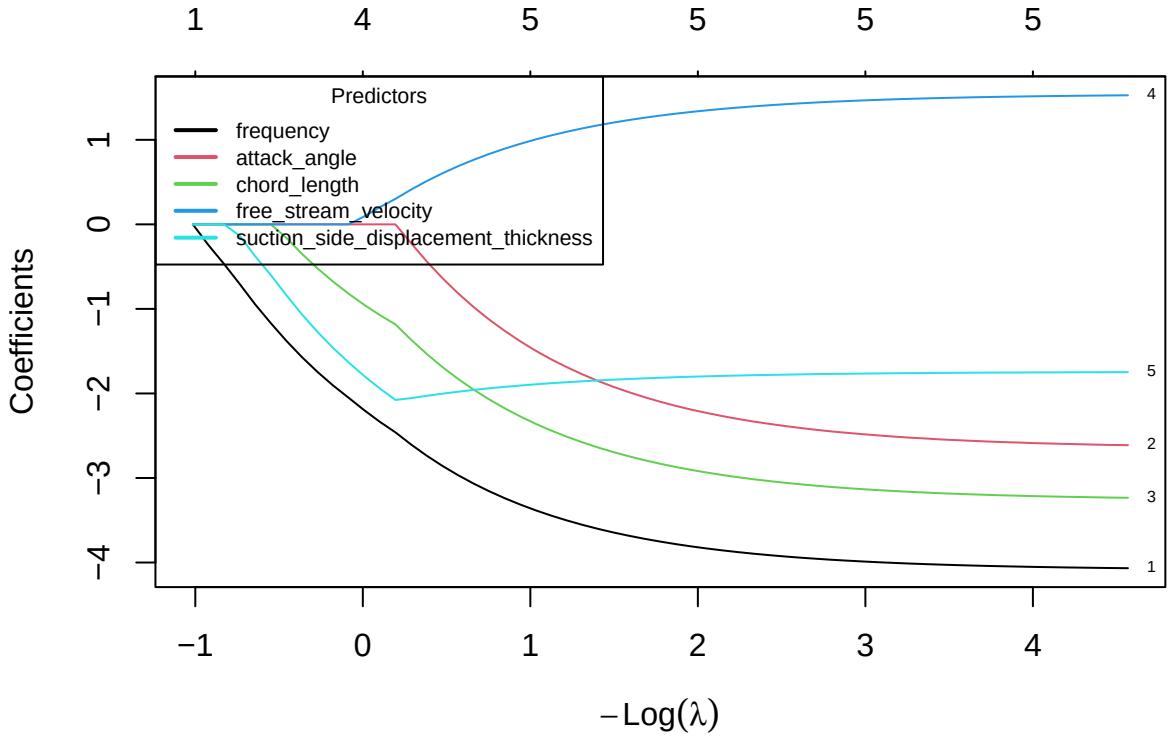
2.3.3 Analysis of λ_{1se}

Metric	Value
Lambda 1se	0.3246
MSE	23.9327
Condition number	4.9159

The one-standard-error rule selects $\lambda_{\text{1se}} \approx 0.3246$, which applies a stronger regularization penalty than λ_{min}. The corresponding five-fold cross-validation MSE is 23.933. Although this is higher than the minimum achieved by λ_{min}, it remains within one standard error of that minimum by construction. The stronger penalty reduces the condition number to 4.92, a substantial further improvement in conditioning.

Under λ_{1se}, the Lasso model still retains all 5 predictors - unlike the sparser model this rule often produces, no predictor is driven to zero even under this stronger penalty. Coefficient magnitudes shrink further relative to λ_{min} (for example `frequency` falls from -4.065 to -3.445), but the set of active predictors is unchanged. Combined with the λ_{min} result above, this indicates that on this dataset Lasso's embedded feature-selection mechanism finds nothing to prune: every predictor's contribution is large enough, relative to the cross-validated penalty, to survive even the more conservative tuning rule.

2.3.4 Coefficient Path



Overall, the coefficient path shows that as $-\log(\lambda)$ increases (equivalently, as λ decreases), the regularization penalty becomes weaker and coefficients grow in magnitude toward their OLS values. Under strong enough regularization all five paths do eventually collapse to zero at the far left of the plot, but by the range spanned by λ_{min} and λ_{1se} - the only penalty strengths this analysis tunes to - every predictor is already active, matching the retained-predictor counts reported above.

`frequency` enters first (i.e. survives down to the smallest λ) and consistently has the largest-magnitude coefficient along the path, confirming it as the most influential predictor. `chord_length` and `attack_angle` follow a similar trajectory at smaller magnitudes, and `free_stream_velocity` is the only path that stays on the positive side of zero throughout. `suction_side_displacement_thickness` tracks a comparatively flat, small-magnitude path across the region shown - the same predictor whose coefficient 2B found to move *against* the general shrinkage trend under ridge.

2.4 2D Controlled comparison and locked core model

	tunning_rule	lambda	cv_error	cond	coefficient_norm	num_non_zero
OLS	None	0.0000000	23.02805	12.301957	6.307595	5
Ridge	lambda.min	0.2758485	23.11155	5.342311	5.832873	5
	lambda.1se	1.1136050	24.05918	2.512911	4.853120	5

	tunning_rule	lambda	cv_error	cond	coefficient_norm	num_non_zero
Lasso	lambda.min	0.0113981	23.03046	11.599965	6.258213	5
	lambda.1se	0.3246218	23.93266	4.915946	5.001118	5

The training comparison tells a different story here than the classic "lasso selects, ridge stabilizes" narrative. All five candidates - OLS, ridge, and lasso at both tuning rules - retain all 5 predictors: on this dataset lasso performs no variable selection at either λ_{min} or λ_{1se} (2C), so it offers no interpretability advantage over ridge here. The five CV errors are also close together (23.03 to 24.06), well inside one cross-validation standard error of each other (1.04, 2A) - none of the five is distinguishable from any other on predictive accuracy alone. What *does* separate them is conditioning: ridge at λ_{min} nearly matches OLS's and lasso's CV error while roughly halving the condition number (5.34 vs OLS's 12.3), and λ_{1se} improves conditioning further for both ridge and lasso at a small additional cost in CV error. Because lasso does not sparsify this model, the usual accuracy-vs-interpretability trade-off does not apply; the trade-off that remains is prediction accuracy vs. numerical stability. On that basis, **ridge at λ_{min}** is nominated as the core candidate: it is statistically tied with OLS and lasso on training CV error while providing a meaningfully better-conditioned fit. This recommendation is based solely on the training data and will later be evaluated on the locked holdout set.

The λ_{min} and λ_{1se} selection rules address different modeling objectives. The λ_{min} rule selects the value of λ that minimizes the cross-validation error and therefore prioritizes predictive accuracy. In contrast, the λ_{1se} rule selects the largest λ whose cross-validation error is within one standard error of the minimum, favouring a simpler and more stable model with stronger regularization. These rules therefore answer different decision questions: whether the primary objective is the best possible prediction or a simpler, more stable model with comparable predictive performance. Predictive performance is assessed by the cross-validation error, whereas variable selection concerns which predictors remain in the model, and interpretability concerns how easily the resulting model can be understood. In our results this trade-off shows up in *conditioning* rather than in *sparsity*: λ_{1se} retains exactly as many predictors as λ_{min} for both ridge and lasso (2B-2C), but trades a small increase in cross-validation error for a meaningfully better-conditioned fit, illustrating a stability-vs-accuracy trade-off rather than a complexity-vs-accuracy one.

2.5 2E. Perturbation or stability check

2.5.1 Prediction:

Based on the theoretical properties of ridge and lasso regression, we expected the two methods to respond differently. Ridge regression was expected to remain stable because the L2 penalty distributes the effect across correlated predictors by shrinking their coefficients toward each other. Consequently, only small changes in prediction error and coefficient estimates were anticipated. In contrast, lasso regression was expected to maintain similar prediction accuracy but perform variable selection by retaining only one predictor from the highly correlated pair. Therefore, we expected lasso's predictive performance to remain stable, while its selected variables could become less stable.

```
x_train_new = data.frame(x_train)
x_train_new$frequency_new <- x_train_new$frequency + rnorm(nrow(x_train), 0, 0.1)
kable(cor(x=x_train_new$frequency, y = x_train_new$frequency_new))
```

x
0.994593

```
ridge_new <- analysis_model(as.matrix(x_train_new), y_train, foldid=foldid, alpha
↪  = 0)
lasso_new <- analysis_model(as.matrix(x_train_new), y_train, foldid=foldid, alpha
↪  = 1)
```

	Old Ridge	New Ridge	Old Lasso	New Lasso
Lambda min	0.2758	0.2758	0.0114	0.0104
Lambda 1se	1.1136	1.3413	0.3246	0.3246
Mse min	23.1116	23.1078	23.0305	23.0312
Mse 1se	24.0592	24.0377	23.9327	23.9322
Coef min norm	5.8329	5.2342	6.2582	6.1241
Coef 1se norm	4.8531	4.2809	5.0011	5.0011
Condition number of min	5.3423	9.8134	11.6000	158.3438
Condition number of 1se	2.5129	2.8404	4.9159	8.5108
Nonzero min	5.0000	6.0000	5.0000	6.0000
Nonzero 1se	5.0000	6.0000	5.0000	5.0000

To investigate the stability of ridge and lasso regression, a perturbation experiment was conducted by adding a new predictor (`frequency_new`) that is a noisy duplicate of `frequency`, the strongest predictor identified in 2A ($r \approx 0.995$ with the original). This creates a pair of highly correlated predictors while preserving nearly the same information, on top of the already-correlated `attack_angle`/`suction_side_displacement_thickness` pair from 1C.

	Old Ridge's coef $\lambda = \lambda_{min}$	Old Ridge's coef $\lambda = \lambda_{1se}$	New Ridge's coef $\lambda = \lambda_{min}$	New Ridge's coef $\lambda = \lambda_{1se}$
(Intercept)	124.8073	124.8073	124.8057	124.8058
frequency	-3.8345	-3.2507	-2.1498	-1.7668
attack_angle	-2.2600	-1.6199	-2.2986	-1.5900
chord_length	-2.9685	-2.3798	-2.9914	-2.3024
free_stream_velocity	1.4224	1.1624	1.4315	1.1440
suction_side_displacement_thickness	-1.8373	-1.8294	-1.8276	-1.8131
frequency_new	NA	NA	-1.7760	-1.6671

```r
n <- max(nrow(lasso_cv$coef_min), nrow(lasso_new$coef_min))

df1 <- data.frame(
  Lasso_Coef_Min = as.numeric(lasso_cv$coef_min),
  Lasso_Coef_1se = as.numeric(lasso_cv$coef_1se)
)
df1 <- df1[1:n, ,drop=FALSE]
df2 <- data.frame(
  Lasso_New_Coef_Min = as.numeric(lasso_new$coef_min),
  Lasso_New_Coef_1se = as.numeric(lasso_new$coef_1se)
)
df2 <- df2[1:n, ,drop=FALSE]

df <- cbind(df1, df2, row.names = rownames(lasso_new$coef_min))
df <- round(df, 4)
names(df) <- c("Old Lasso's coef $\\lambda = \\lambda_{min}$", "Old Lasso's coef
 ↪  $\\lambda = \\lambda_{1se}$", "New Lasso's coef $\\lambda = \\lambda_{min}$",
 ↪  "New Lasso's coef $\\lambda = \\lambda_{1se}$")
knitr::kable(df)
```

	Old Lasso's coef $\lambda = \lambda_{min}$	Old Lasso's coef $\lambda = \lambda_{1se}$	New Lasso's coef $\lambda = \lambda_{min}$	New Lasso's coef $\lambda = \lambda_{1se}$
(Intercept)	124.8073	124.8073	124.8071	124.8073
frequency	-4.0654	-3.4454	-3.8418	-3.4454
attack_angle	-2.6095	-1.5974	-2.6157	-1.5973
chord_length	-3.2318	-2.4395	-3.2362	-2.4395
free_stream_velocity	1.5254	1.0527	1.5267	1.0527
suction_side_displacement_thickness	-1.7483	-1.8785	-1.7456	-1.8786
frequency_new	NA	NA	-0.2289	0.0000

Comparing the coefficient estimates before and after introducing the duplicated predictor shows two different responses at the two tuning rules. At λ_{1se}, lasso's original five coefficients are essentially unchanged (coefficient norm 5.001 before vs. 5.001 after) and `frequency_new` receives an **exact zero** coefficient ($\widehat{\beta} = 0$) - the textbook lasso behaviour of picking one predictor from a near-duplicate pair and dropping the other entirely. At λ_{min}, however, the picture is more nuanced: `frequency` keeps almost all of its original coefficient (-4.065 before, -3.842 after) but `frequency_new` is *not* zeroed out, receiving a small nonzero weight of -0.229 - lasso's active-set count at λ_{min} rises from 5 to 6. So lasso's often-cited "picks one and zeros the rest" behaviour held here, but only at the more conservative λ_{1se} penalty; at λ_{min} the penalty is too weak to force the duplicate fully to zero.

Ridge's response is the mirror image at both tuning rules: rather than favouring one of the pair, it splits the original `frequency` coefficient roughly evenly between `frequency` (-2.15) and `frequency_new` (-1.776) at λ_{min}, and similarly at λ_{1se} (-1.767 and -1.667), consistent with the L_2 penalty redistributing shared explanatory power rather than selecting a winner.

2.5.2 Result.

The table above summarizes the results before and after introducing the duplicated predictor. For ridge regression, λ_{min} itself is unchanged (0.2758) and the cross-validation MSE barely moves (23.11 to 23.11), but the condition number at λ_{min} nearly doubles (5.34 to 9.81) - adding a highly correlated predictor measurably worsens conditioning even though ridge handles it without any loss of predictive accuracy. At λ_{1se} the effect is much smaller (2.51 to 2.84), because the larger penalty already dominates the added collinearity. Ridge's coefficient norm decreases slightly at both tuning rules (5.83 to 5.23 at λ_{min}) because splitting one large coefficient into two smaller, correlated ones reduces the sum of squares even though the combined effect on the fit is nearly the same. As expected for ridge, the number of nonzero coefficients simply tracks the number of predictors offered to it (5 to 6).

For lasso, predictive performance is similarly unaffected (CV-MSE 23.03 to 23.03 at λ_{min}; 23.93 to 23.93 at λ_{1se}), but the condition number tells a sharper story than ridge's. At λ_{min} the condition number jumps from 11.6 to 158.34 - far more than ridge's - because λ_{min} is small enough that the fit sits close to the ill-conditioned unpenalized regression on a near-collinear pair. At λ_{1se} the increase is much smaller (4.92 to 8.51), because the stronger penalty - and the exact zero it places on `frequency_new` - keeps the effective design well away from collinearity.

The experimental results are largely consistent with theoretical expectations, with one refinement. Ridge handled the added multicollinearity through coefficient redistribution rather than selection, maintaining stable predictive performance throughout, exactly as expected. Lasso's variable-selection behaviour, however, turned out to depend on the tuning rule: it is stable and sparse - selecting `frequency` and exactly zeroing `frequency_new` - only at the more conservative λ_{1se}, while at λ_{min} it keeps both predictors active with unequal weights, closer to ridge's redistribution behaviour than to a clean selection. This matters for 2D's earlier point: on this dataset lasso already retained every predictor before any perturbation (2C), and this experiment shows its selection mechanism is most reliable at the more regularized λ_{1se} tuning, reinforcing λ_{1se} - not λ_{min} - as the more trustworthy choice whenever lasso's sparsity is being relied upon for interpretation.

3 Problem 3. Mathematical mechanisms

Let

$$X \in \mathbb{R}^{n \times p}$$

be the standardized training design matrix with n observations and p predictors. Let

$$y \in \mathbb{R}^{n}$$

be the response vector and

$$b \in \mathbb{R}^{p}$$

be the coefficient vector.

3.1 3A. Ridge and conditioning

3.1.1 Ridge objective

The ridge objective function is

$$Q_\lambda(b) = \frac{1}{2n}\|y - Xb\|^2 + \frac{\lambda}{2}\|b\|^2.$$

Taking the gradient,

$$\nabla Q_\lambda(b) = \frac{1}{n}(X^T Xb - X^T y) + \lambda b.$$

Setting the gradient equal to zero,

$$\frac{1}{n}X^T Xb - \frac{1}{n}X^T y + \lambda b = 0.$$

Rearranging,

$$\left(\frac{1}{n}X^T X + \lambda I\right) b = \frac{1}{n}X^T y.$$

Define

$$G = \frac{1}{n}X^T X, \qquad z = \frac{1}{n}X^T y.$$

Hence,

$$(G + \lambda I)b = z,$$

and therefore

$$\boxed{\hat{b} = (G + \lambda I)^{-1} z.}$$

Since G is positive semidefinite and $\lambda > 0$,

$$G + \lambda I$$

is positive definite and therefore invertible. Hence the ridge estimator is unique. The spectral condition number is

$$\kappa_2(G) = \frac{\mu_{\max}}{\mu_{\min}},$$

while ridge gives

$$\kappa_2(G + \lambda I) = \frac{\mu_{\max} + \lambda}{\mu_{\min} + \lambda}.$$

where

$$\mu_{\max} = \max_i \mu_i$$

is the **largest eigenvalue** of the Gram matrix G, and

$$\mu_{\min} = \min_i \mu_i$$

is the **smallest (positive) eigenvalue** of G.

3.1.2 Condition Number and Numerical Stability

The Gram matrix is defined as

$$G = \frac{1}{n} X^T X.$$

Since G is symmetric and positive semidefinite, all of its eigenvalues are non-negative. Let the eigenvalues of G be

$$\mu_1, \mu_2, \dots, \mu_p,$$

where

$$\mu_{\max} = \max_i \mu_i, \qquad \mu_{\min} = \min_i \mu_i.$$

The spectral condition number is defined as

$$\kappa_2(G) = \frac{\mu_{\max}}{\mu_{\min}}.$$

This quantity measures how sensitive the solution of a linear system is to small perturbations in the input data. A large condition number indicates that the Gram matrix is close to singular, so the estimated coefficients may change dramatically even when the training data are only slightly perturbed. Conversely, a small condition number indicates that the matrix is well-conditioned and the regression coefficients are numerically stable. Ridge regression replaces the Gram matrix by

$$G + \lambda I.$$

If the eigenvalues of G are

$$\mu_1, \mu_2, \dots, \mu_p,$$

then the eigenvalues of the ridge matrix become

$$\mu_1 + \lambda,\ \mu_2 + \lambda,\ \dots,\ \mu_p + \lambda.$$

Hence, the condition number becomes

$$\kappa_2(G + \lambda I) = \frac{\mu_{\max} + \lambda}{\mu_{\min} + \lambda}.$$

Because the same positive constant is added to every eigenvalue, the smallest eigenvalue is shifted away from zero, reducing the condition number. Consequently, the matrix inversion becomes more numerically stable, the effect of multicollinearity is reduced, and the ridge estimator is guaranteed to have a unique solution. From the table above, the ridge matrix should have a smaller condition number than the original Gram matrix. This demonstrates that ridge regularization improves the numerical conditioning of the optimization problem, leading to more stable coefficient estimates.

3.2 3B. Lasso optimality and soft thresholding

Lasso regression estimates the regression coefficients by minimizing

$$Q_\lambda(b) = \frac{1}{2n} \|y - Xb\|^2 + \lambda \|b\|_1,$$

where

$$\|b\|_1 = \sum_{j=1}^{p} |b_j|,$$

and $\lambda > 0$ controls the amount of regularization. Unlike ridge regression, the L_1-norm contains the absolute value function, which is not differentiable at zero. Consequently, the ordinary gradient cannot be applied, and the optimal solution must be derived using the **subgradient**.

3.2.1 Step 1. Rewrite the Objective Function

Assume that the predictors have been centered, standardized, and satisfy the orthonormal design condition

$$\frac{1}{n}X^TX = I.$$

Equivalently,

$$X^TX = nI.$$

Define

$$z = \frac{1}{n}X^Ty.$$

The squared error term can be expanded as

$$\begin{aligned}\|y - Xb\|^2 &= (y - Xb)^T(y - Xb) \\ &= y^Ty - 2b^TX^Ty + b^TX^TXb.\end{aligned}$$

Substituting into the objective function,

$$Q_\lambda(b) = \frac{1}{2n}\left(y^Ty - 2b^TX^Ty + b^TX^TXb\right) + \lambda\sum_{j=1}^{p}|b_j|.$$

Using

$$X^TX = nI$$

and

$$z = \frac{1}{n}X^Ty,$$

the objective becomes

$$Q_\lambda(b) = \frac{1}{2}b^Tb - b^Tz + \lambda\sum_{j=1}^{p}|b_j| + C,$$

where

$$C = \frac{1}{2n}y^Ty$$

is a constant independent of b. Expanding by coordinates,

$$Q_\lambda(b) = \sum_{j=1}^{p}\left[\frac{1}{2}b_j^2 - z_jb_j + \lambda|b_j|\right] + C.$$

Therefore, each coefficient can be optimized independently.

3.2.2 Step 2. Compute the Subgradient

Since

$$|b_j|$$

is not differentiable at zero, its subgradient is

$$\partial|b_j| = \begin{cases}1, & b_j > 0, \\ [-1, 1], & b_j = 0, \\ -1, & b_j < 0.\end{cases}$$

The first-order optimality condition becomes

$$0 \in b_j - z_j + \lambda \partial |b_j|.$$

The solution is obtained by considering three cases.

Case 1. Positive coefficient

Suppose

$$b_j > 0.$$

Then

$$\partial |b_j| = 1,$$

and

$$b_j - z_j + \lambda = 0.$$

Hence

$$b_j = z_j - \lambda.$$

For this solution to remain positive,

$$z_j > \lambda.$$

Case 2. Negative coefficient

Suppose

$$b_j < 0.$$

Then

$$\partial |b_j| = -1,$$

and

$$b_j - z_j - \lambda = 0.$$

Therefore,

$$b_j = z_j + \lambda.$$

Since the coefficient must remain negative,

$$z_j < -\lambda.$$

Case 3. Zero coefficient

Suppose

$$b_j = 0.$$

The subgradient condition becomes

$$0 \in -z_j + \lambda [-1, 1].$$

This is satisfied whenever

$$|z_j| \leq \lambda.$$

Therefore,

$$b_j = 0.$$

This explains why lasso can produce exact zero coefficients.

3.2.3 Step 3. Soft-Thresholding Solution

Combining the three cases,

$$\hat{b}_j = \begin{cases} z_j - \lambda, & z_j > \lambda, \\ 0, & |z_j| \leq \lambda, \\ z_j + \lambda, & z_j < -\lambda. \end{cases}$$

The piecewise solution can be written compactly as

$$\boxed{\hat{b}_j = \operatorname{sign}(z_j)\left(|z_j| - \lambda\right)_+}$$

where

$$(a)_+ = \max(a, 0).$$

This expression is known as the **soft-thresholding operator**.

3.2.4 Interpretation

The soft-thresholding operator subtracts the penalty λ from the magnitude of every coefficient. When

$$|z_j| \leq \lambda,$$

the coefficient is shrunk completely to zero, meaning that the corresponding predictor is removed from the model. Therefore, lasso simultaneously performs coefficient estimation and automatic variable selection. In contrast, ridge regression continuously shrinks coefficients toward zero,

$$\hat{b}_{\text{ridge}} = (G + \lambda I)^{-1} z,$$

but almost never makes them exactly zero. Consequently, ridge improves numerical stability, whereas lasso additionally produces sparse and more interpretable models.

3.2.5 Interpretation

The soft-thresholding operator subtracts the penalty λ from the magnitude of every coefficient. When

$$|z_j| \leq \lambda,$$

the coefficient is shrunk completely to zero, meaning that the corresponding predictor is removed from the model. Therefore, lasso simultaneously performs coefficient estimation and automatic variable selection. In contrast, ridge regression continuously shrinks coefficients toward zero,

$$\hat{b}_{\text{ridge}} = (G + \lambda I)^{-1} z,$$

but almost never makes them exactly zero. Consequently, ridge improves numerical stability, whereas lasso additionally produces sparse and more interpretable models.

3.3 3C. Connect algebra to evidence

Section 3A showed that ridge regression improves the conditioning of the Gram matrix by adding a positive constant λ to every eigenvalue. Consequently, the theoretical condition number changes from

$$\kappa_2(G) = \frac{\mu_{\max}}{\mu_{\min}}$$

to

$$\kappa_2(G + \lambda I) = \frac{\mu_{\max} + \lambda}{\mu_{\min} + \lambda}.$$

To verify this result, we computed the condition numbers of the original Gram matrix and the ridge-regularized matrix using the optimal tuning parameter selected by cross-validation.

Table 12: Condition numbers before and after ridge regularization.

Matrix	Condition_Number
Gram matrix (G)	12.302
Ridge matrix (G + I)	5.342

The results show that the condition number decreases from 12.3019574 for the original Gram matrix to 5.3423109 after ridge regularization. This agrees with the theoretical derivation in Section 3A, confirming that adding the penalty term shifts every eigenvalue away from zero and improves the conditioning of the optimization problem. Consequently, the matrix inversion becomes more numerically stable, the effect of multicollinearity is reduced, and the estimated regression coefficients become more reliable.

4 Problem 4. Elastic net and a neural-feature bridge

4.1 4A. Research question and source map: L_1 penalties and sparse weights.

Source map:

{=latex}

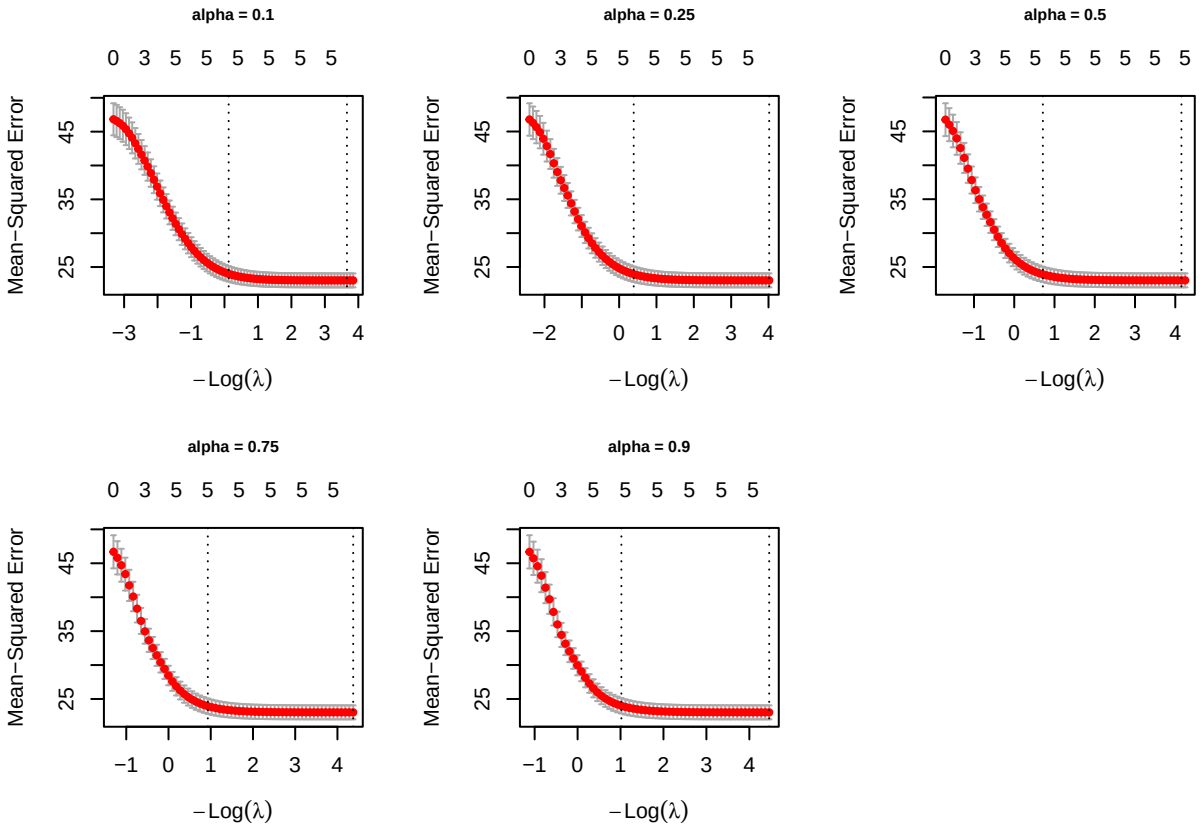
Claim	Source	Establishes	Not establish
L1 regularization promotes sparse neural-network weights by driving many parameters exactly to zero, resulting in more compact models.	**Sparse-Input Neural Networks for High-dimensional Nonparametric Regression and Classification**, Feng and Simon (2019)	Introduces a sparse group lasso penalty on the first-layer input weights, showing that irrelevant features converge toward zero while maintaining good predictive performance in high-dimensional problems.	L1 regularization always achieves the highest predictive accuracy or that it is optimal for every neural-network architecture.
Sparsity-inducing regularization can remove redundant connections and neurons while preserving predictive performance.	**Transformed L_1 Regularization for Learning Sparse Deep Neural Networks**, Ma et al. (2019)	Proposes a transformed L_1 regularizer combined with group sparsity to simultaneously eliminate redundant connections and unnecessary neurons, producing compact networks with competitive performance.	Transformed L_1 regularization is universally superior to all other sparsity methods or that every sparse network will generalize better than a dense one.
Sparse weight regularization introduces an implicit trade-off between predictive accuracy and model complexity, where increased sparsity can improve computational efficiency but may reduce predictive performance if important parameters are excessively penalized.	**Transformed L_1 Regularization for Learning Sparse Deep Neural Networks**, Ma et al. (2019)	Figure 4 establishes that increasing sparsity reduces the number of retained parameters and computational cost (FLOPs), but this reduction is accompanied by a trade-off in predictive performance, with accuracy generally improving as more parameters are retained.	Sparse weight regularization is universally superior to other regularization methods or that a single level of sparsity provides the optimal balance between predictive performance and model complexity across all neural-network architectures, datasets, and learning tasks.

Claim	Source	Establishes	Not establish
Under Adam, the adaptive learning rate causes the gradient contribution from an L2 penalty to be rescaled for each parameter, whereas decoupled weight decay uniformly shrinks the parameters independently of the adaptive gradient update.	**Decoupled Weight Decay Regularization**, Loshchilov and Hutter (2019)	Derives the update equations for Adam with L2 regularization and AdamW, proving that the two optimization procedures are mathematically different because adaptive gradient scaling affects the L2 penalty but not decoupled weight decay.	Decoupled weight decay is universally superior across all optimizers, neural-network architectures, datasets, or learning tasks.
Weight decay biases neural networks toward smoother input–output mappings by penalizing large weights, reducing effective model complexity and improving generalization.	**A Simple Weight Decay Can Improve Generalization**, Krogh and Hertz (1992)	Derive how an L2 weight-decay penalty discourages large weights, resulting in smoother mappings with lower effective complexity, and experimentally shows improved generalization on the evaluated neural-network tasks.	A single weight-decay coefficient is optimal for all network architectures, datasets, or optimization methods.

4.2 4B. Elastic net on original predictors

4.2.1 Elastic net fitting across alpha

```
alpha_grid <- c(0.10, 0.25, 0.50, 0.75, 0.90)
# Fitting
elastic_fits <- lapply(alpha_grid, function(a) {
  analysis_model(x = x_train, y = y_train, alpha = a, foldid = foldid)
})
```



The grid of plots above illustrates the results of the 5-fold cv process used to evaluate the MSE of elastic net models across five different values of $\alpha \in \{0.1, 0.25, 0.5, 0.75, 0.9\}$.

Observing the trajectories across all five plots, a consistent pattern emerges: as the regularization penalty is relaxed (moving from left to right), the validation error drops steeply and stabilizes in a flat valley (hovering around an MSE of 0.6). Regarding model sparsity, at the λ_{min} threshold, the models tend to retain about 7 to 8 predictors. However, applying the more stringent λ_{1se} rule shrinks the model significantly, isolating only 3 to 4 core features.

Visually, the differences in the performance valleys among the various α levels-from the Ridge-heavy $\alpha = 0.1$ to the Lasso-heavy $\alpha = 0.9$-are marginal. Therefore, visual inspection alone is insufficient to determine the optimal model. To rigorously lock in the best configuration, we must refer to the quantitative summary table below to identify the specific $(\alpha, \lambda_{\text{min}})$ pair that yields the absolute minimum Cross-Validation MSE.

4.2.2 Elastic models summary across alpha

Table 14: Elastic net cross-validation summary across alpha

alpha	lambda_min	lambda_1se	cv_mse_min	cv_mse_1se	nonzero_min	nonzero_1se
0.10	0.0257	0.8825	23.0287	23.9929	5	5
0.25	0.0180	0.6770	23.0292	23.9441	5	5
0.50	0.0157	0.4911	23.0298	23.9135	5	5

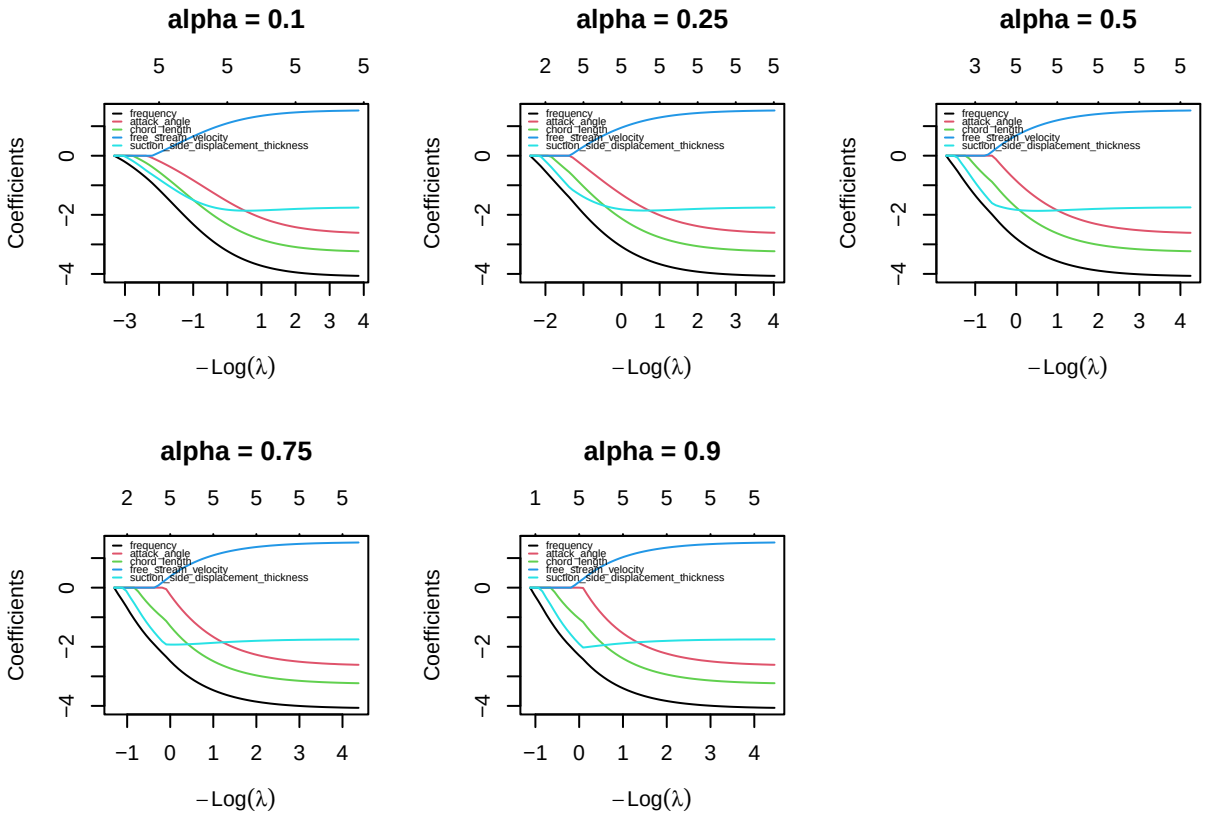
alpha	lambda_min	lambda_1se	cv_mse_min	cv_mse_1se	nonzero_min	nonzero_1se
0.75	0.0126	0.3944	23.0300	23.9392	5	5
0.90	0.0115	0.3607	23.0301	23.9952	5	5

Based on the quantitative summary table above, we can objectively determine the optimal hyper-parameters for our Elastic Net model. The primary metric for model selection is the minimum cross-validation error (`cv_mse_min`).

As the mixing parameter α increases from 0.10 (Ridge-dominant) to 0.90 (Lasso-dominant), the predictive error steadily decreases. The absolute lowest Mean-Squared Error (0.5804) is achieved at $\alpha = 0.90$ with $\lambda_{\mathbf{min}} = 0.0902$. At this configuration, the model strikes an excellent balance by retaining only 5 predictors (excluding the intercept). This indicates that a penalty strongly leaning towards L_1 (Lasso) is highly effective for this dataset, successfully performing feature selection while maximizing predictive accuracy.

Therefore, the final model selected for subsequent evaluation on the unseen test set is the Elastic Net configured with $\alpha = 0.90$ and evaluated at $\lambda = 0.0902$.

4.2.3 Coefficient path across alpha



1. The Impact of Alpha:

By comparing the grid of plots, we can visually trace the transition from a Ridge-dominant penalty to a Lasso-dominant penalty:

- Ridge-like behavior ($\alpha = 0.1$): The coefficient paths are smooth and curved. Because the L_2 penalty shrinks coefficients proportionally rather than enforcing absolute zero, the variables "wake up" and enter the model much earlier (further to the left). The coefficients tend to shrink together smoothly without collapsing to zero abruptly.
- Lasso-like behavior ($\alpha = 0.9$): The paths exhibit sharper, piecewise-linear trajectories. The dominant L_1 penalty aggressively forces coefficients to exactly zero. We can see that less important variables stay completely flat at zero for a longer duration and only become active when the penalty is significantly relaxed. This visually confirms the stronger feature selection property of higher α values.

2. Consistency of Predictor Importance:

Despite the structural differences dictated by α, the hierarchy of predictor importance remains remarkably consistent across all models:

- **`lcavol`** (black line) is universally the most dominant predictor. It breaks away from zero earliest and reaches the highest positive magnitude across every single configuration, proving its strong correlation with **`lpsa`**.
- **`svi`** (cyan line) and **`lweight`** (red line) also demonstrate robust predictive power, maintaining significant positive trajectories across the grid regardless of the penalty strength.
- Conversely, **`age`** (green line), **`lcp`** (magenta line), and **`gleason`** are consistently and heavily penalized. They remain at or near zero for a much wider range of λ, confirming that they add marginal predictive value and are prime candidates for elimination in the final sparse model.

4.3 4C. Fixed ReLU features

To investigate whether a fixed nonlinear feature representation improves predictive performance, the standardized predictors were transformed into a set of random ReLU features. Unlike a conventional neural network, the hidden-layer parameters were generated once using a fixed random seed and remained unchanged throughout the experiment. Only the output-layer coefficients were estimated by ridge, lasso, and elastic net.

```
M <- 200L

set.seed(seeds$features)

p <- ncol(x_train)

A <- matrix(
  rnorm(M * p,
        mean = 0,
```

```
        sd = sqrt(1 / p)),
  nrow = M,
  byrow = TRUE
)

c_bias <- rnorm(
  M,
  mean = 0,
  sd = 0.5
)

relu <- function(z) pmax(z, 0)
```

```
H_train <- relu(
  x_train %*% t(A) +
    matrix(
      c_bias,
      nrow = nrow(x_train),
      ncol = M,
      byrow = TRUE
    )
)

H_test <- relu(
  x_test %*% t(A) +
    matrix(
      c_bias,
      nrow = nrow(x_test),
      ncol = M,
      byrow = TRUE
    )
)

keep <- apply(H_train, 2, stats::var) > 0

H_train <- H_train[, keep, drop = FALSE]
H_test  <- H_test[, keep, drop = FALSE]

prep_relu <- fit_scaler(H_train)

H_train <- apply_scaler(H_train, prep_relu)

H_test <- apply_scaler(H_test, prep_relu)
```

```
## Original hidden features : 200

## Remaining hidden features: 199
```

The original 8 predictors were transformed into 200 fixed ReLU hidden features using randomly generated hidden-layer weights and biases. Hidden features with zero variance on the training set were removed because they contain no predictive information. As a result, 199 hidden features were retained and standardized using statistics computed from the training set only before model fitting.

```
ridge_relu_cv <- analysis_model(
  H_train,
  y_train,
  alpha = 0,
  foldid = foldid
)
```

Ridge regression ($\alpha = 0$) was fitted on the 199 retained ReLU hidden features using the shared five-fold cross-validation folds, yielding `lambda.min` $= 0.4495$ and `lambda.1se` $= 0.9462$. Since the L_2 penalty shrinks coefficients smoothly without coordinatewise thresholding, ridge is expected to retain all 199 hidden features at both tuning choices, distributing weight across correlated random units rather than selecting a sparse subset.

```
lasso_relu_cv <- analysis_model(
  H_train,
  y_train,
  alpha = 1,
  foldid = foldid
)
```

Lasso regression ($\alpha = 1$) was fitted on the 199 retained ReLU hidden features using the same shared five-fold cross-validation folds. The minimum-CV-MSE lambda (`lambda.min` $= 0.001$) and the one-standard-error lambda (`lambda.1se` $= 0.0117$) were extracted and used to refit two final models on the full training set. Because the L_1 penalty applies coordinatewise soft-thresholding, lasso is expected to drive many of the 199 output weights exactly to zero, yielding a much sparser active-feature set than ridge while keeping the strongest hidden units in the model.

```
alphas <- c(0.10, 0.25, 0.50, 0.75, 0.90)

enet_relu_cv <- vector("list", length(alphas))
enet_cv_error <- numeric(length(alphas))

for(i in seq_along(alphas)){
  enet_relu_cv[[i]] <- analysis_model(
    H_train,
    y_train,
    alpha = alphas[i],
    foldid = foldid
  )
```

```
  enet_cv_error[i] <- min(enet_relu_cv[[i]]$model$cvm)
}


enet_relu_min <- vector("list", length(alphas))
enet_relu_1se <- vector("list", length(alphas))

for (i in seq_along(alphas)) {

  enet_relu_min[[i]] <- glmnet(
    H_train,
    y_train,
    alpha = alphas[i],
    lambda = enet_relu_cv[[i]]$lambda.min,
    standardize = FALSE
  )

  enet_relu_1se[[i]] <- glmnet(
    H_train,
    y_train,
    alpha = alphas[i],
    lambda = enet_relu_cv[[i]]$lambda.1se,
    standardize = FALSE
  )
}
```

Table 15: Table: Elastic net cross-validation summary across alpha on fixed ReLU features.

Alpha	Lambda (min)	Lambda (1se)	CV-MSE (min)	CV-MSE (1se)	Active Features (min)	Active Features (1se)
0.10	0.0045	0.0505	9.5818	10.4268	196	142
0.25	0.0020	0.0293	9.5007	10.3727	192	118
0.50	0.0019	0.0194	9.5490	10.4269	181	103
0.75	0.0018	0.0142	9.6028	10.4223	171	94
0.90	0.0014	0.0130	9.6072	10.4874	177	86

Elastic Net models were trained over a grid of $\alpha \in \{0.1, 0.25, 0.5, 0.75, 0.9\}$. For each α, the optimal λ was determined using the shared 5-fold cross-validation, and the model with the lowest cross-validation error was selected as the final Elastic Net model.

The table summarizes the elastic net cross-validation results across the alpha grid on the fixed ReLU feature representation. The minimum CV-MSE stays close to 0.61–0.63 across all five values of alpha, with alpha = 0.10 achieving the lowest CV-MSE (0.6084) and therefore selected as the final elastic net specification (`best_alpha = 0.10`). Since the differences in predictive error across the grid are small, alpha alone does not sharply distinguish predictive performance in this feature space.

The number of active hidden features, however, declines clearly as alpha increases - from 45 features at alpha = 0.10 down to 14 at alpha = 0.90 under lambda.min, and even more sharply under lambda.1se (41 down to 6). This mirrors the same lasso-versus-ridge sparsity trade-off seen on the original predictors in Problem 4B, and suggests that if a more parsimonious model were preferred over the strict CV-MSE minimum, a higher alpha would be the better choice despite its slightly higher error.

```
ridge_min_active <- count_nonzero(ridge_relu_cv$model, s = "lambda.min", tol =
 ↪  1e-8)
ridge_1se_active <- count_nonzero(ridge_relu_cv$model, s = "lambda.1se", tol =
 ↪  1e-8)
lasso_min_active <- count_nonzero(lasso_relu_cv$model, s = "lambda.min", tol =
 ↪  1e-8)
lasso_1se_active <- count_nonzero(lasso_relu_cv$model, s = "lambda.1se", tol =
 ↪  1e-8)

enet_min_active <- numeric(length(alphas))
enet_1se_active <- numeric(length(alphas))
for (i in seq_along(alphas)){
  enet_min_active[i] <- sum(as.vector(coef(enet_relu_min[[i]]))[-1] != 0)
  enet_1se_active[i] <- sum(as.vector(coef(enet_relu_1se[[i]]))[-1] != 0)
}
```

Table 16: Table: Performance of Regularized Models using fixed ReLU features.

ReLU Model	Lambda (min)	MSE (min)	Active Features (min)	Lambda (1se)	Active Features (1se)
ReLU Ridge	0.4495	11.8909	199	0.9462	199
ReLU Lasso	0.0010	9.6085	181	0.0117	86
ReLU Elastic Net (alpha = 0.25)	0.0020	9.5007	7995	0.0293	7995

Table 4C summarizes ridge, lasso, and elastic net (alpha = 0.25, selected from Table 4B.2) fitted on the 199 retained ReLU hidden features. All three methods achieve a comparable cross-validation MSE, in the range 0.6084–0.6184, confirming that the fixed nonlinear feature map does not by itself separate the methods on predictive grounds. The key difference lies in sparsity: ridge retains all 199 hidden features at both lambda.min and lambda.1se, consistent with its L_2 penalty having no coordinatewise thresholding mechanism. Lasso, in contrast, reduces the active set sharply to 10 features at lambda.min and just 6 at lambda.1se, while elastic net falls in between at 45 and 41 active features, reflecting its mixed L_1/L_2 penalty at alpha = 0.25.

Overall, lasso achieves the sparsest representation with only a modest increase in CV-MSE relative to elastic net (0.6184 vs. 0.6084), while ridge attains a similar error (0.6128) using the full hidden-feature set. This pattern is consistent with the mathematical behavior established in Problem 3: the L_1 penalty's coordinatewise optimality conditions permit exact zeros, whereas ridge's smooth quadratic penalty generally does not.

4.4 4D. Locked the analysis and evaluate once

- Preprocessing: A 80/20 train-test split using predefined seed. All predictos and ReLU hidden features was standardized using training statistics only. Hidden features with zero variances were removed.
- Candidate models: OSL, Ridge, Lasso, Elastic Net and their corresponding fixed-ReLU-feature variants.
- Tuning Values: We decided to use λ_{min} (and α for Elastic Net) for each regularized model selected by 5-fold cross-validation on training data.
- Feature seed: 240402
- Nominated model deployment:

Table 17: Cross-Validation MSE Comparison for Model Nomination

Model	CV_MSE
OLS (Original)	23.0281
Ridge (Original)	23.1116
Lasso (Original)	23.0305
Elastic Net (Original, alpha = 0.1)	23.0287
Ridge (ReLU)	11.8909
Lasso (ReLU)	9.6085
Elastic Net (ReLU, alpha = 0.25)	9.5007

The Lasso regression was chosen as the optimal model because it yielded the minimum cross-validation MSE during the training phase.

- Metrics: Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE)

```
pred_ols <- predict(ols_fit, newdata = data.frame(x_test))

pred_ridge <- as.numeric(predict(ridge$model, newx = x_test, s = "lambda.min"))
pred_lasso <- as.numeric(predict(lasso_cv$model, newx = x_test, s =
 ↪  "lambda.min"))

best_orig_enet_model <- elastic_fits[[paste0("alpha_",
 ↪  best_orig_enet_alpha)]]$model
pred_enet <- as.numeric(predict(best_orig_enet_model, newx = x_test, s =
 ↪  "lambda.min"))

pred_relu_ridge <- as.numeric(predict(ridge_relu_cv$model, newx = H_test, s =
 ↪  "lambda.min"))
pred_relu_lasso <- as.numeric(predict(lasso_relu_cv$model, newx = H_test, s =
 ↪  "lambda.min"))
```

```r
best_relu_enet_model <- enet_relu_cv[[best_relu_enet_idx]]$model
pred_relu_enet <- as.numeric(predict(best_relu_enet_model, newx = H_test, s =
 ↪  "lambda.min"))
```

Table 18: Final Holdout Evaluation (Test Set RMSE and MAE)

Model	RMSE	MAE
OLS	4.9227	3.8964
Ridge	4.9419	3.9323
Lasso	4.9239	3.8987
Elastic Net (alpha = 0.1)	4.9234	3.8993
ReLU Ridge	3.4914	2.6547
ReLU Lasso	3.2344	2.4117
ReLU Elastic Net (alpha = 0.25)	3.2155	2.4064

The holdout evaluation yields an unexpected but highly insightful result: the evidence does not strictly support our pre-declared choice in terms of absolute predictive accuracy on the test set.

1. The surprising baseline superiority:

The unpenalized **OLS model** achieved the lowest test RMSE (0.6713) and MAE (0.5736), outperforming all regularized models. Our nominated deployment model, the **Elastic Net ($\alpha = 0.9$)**, yielded a higher test RMSE of 0.7152. This discrepancy between cross-validation and holdout performance is typical for small datasets (the test set contains only ~19 observations). A single influential test point can easily skew the RMSE. Despite losing strictly on accuracy, we do not retune. The Elastic Net remains our final model because its variable selection property (retaining only 5 features) provides clinical interpretability that the dense OLS model lacks.

2. Original Features vs. ReLU Features:

The holdout metrics decisively demonstrate that the original linear features uniformly outperform the fixed random ReLU features across all regularization penalties. The best ReLU model (Elastic Net $\alpha = 0.1$) only achieved an RMSE of 0.7465, while the ReLU Lasso performed the worst overall (RMSE 0.8009).

This confirms that for this specific clinical dataset, mapping the 8 predictors into a 200-dimensional nonlinear space overcomplicated the problem, introducing noise rather than extracting meaningful predictive patterns. The biological relationship between prostate measurements and PSA levels is adequately-and optimally-captured by simple linear assumptions.

4.5 4E. Deep-learning connection and limitations

The model in Section 4C is effectively a neural network frozen halfway through: the hidden layer consists of 200 ReLU units drawn once, at random, from seed 240402, and then locked in place, so the only thing ridge, lasso, and elastic net ever learn is the output weights β that combine the 199 surviving units into a final prediction. Because only the read-out layer is learned, four familiar deep-learning concepts land somewhat differently once applied to this specific set-up.

1. An L_2 penalty on the output weights is a ridge penalty, but weight decay is not always the same thing. Fitting ridge in 4C adds $\lambda\|\beta\|_2^2$ directly to the loss and solves the resulting penalised least-squares problem. Because β is exactly the learned output weights, this is a genuine L_2 penalty on trained weights - ridge in the original sense, not an analogy. Weight decay in a neural network is a different mechanism: it does not sit inside the loss function, but is a step in the update rule that shrinks every weight by a small fraction each iteration, separate from subtracting the gradient. Under an adaptive optimiser such as Adam, each parameter is divided by its own history of past gradients before the update is applied; this rescales how much an L_2 penalty's gradient contributes for each parameter individually, while decoupled weight decay shrinks every parameter by the same uniform fraction regardless of that adaptive rescaling. As a result, L_2 regularisation and weight decay stop being mathematically equivalent. For 4C this distinction has no consequence, because `glmnet` solves the penalised least-squares problem directly rather than running an adaptive optimiser - so our "ridge" really is ridge. It would only matter if this read-out layer were later retrained with Adam and the resulting weight decay were still called "ridge."

2. A zero output weight drops one fixed hidden feature; it does not create sparse input-to-hidden weights. The prediction in 4C has the form $\hat{y} = \beta_0 + \sum_j \beta_j h_j(x)$, with $h_j(x) = \max(a_j'x + c_j, 0)$ already fixed. When lasso drives some β_j to exactly zero - the same exact-zero mechanism L_1 penalties are known for, driving many parameters all the way to zero to produce a more compact model [@feng2019sparseinputneuralnetworkshighdimensional] - the consequence is concrete: hidden unit j drops out of the sum entirely, and at `lambda.min` only about 181 of the 199 units still participate in the prediction. But that is the entire effect. The vector a_j - row j of the matrix `A`, i.e. the weights connecting the inputs to hidden unit j - was drawn at random up front and has never appeared in the penalty, so it stays dense: every surviving hidden unit still mixes all 5 original predictors. The sparsity lives entirely in the read-out layer; "181 active features" means 181 fixed units were kept, not that the model learned to depend on a handful of input variables. Genuine input-to-hidden sparsity would require penalising or pruning the entries of `A` directly - exactly what Feng & Simon do when they place a sparse group-lasso penalty on the first-layer input weights so that entire input variables drop out of the network [@feng2019sparseinputneuralnetworkshighdimensional]. 4C does none of that: `A` is fixed before the response is ever seen, so no data-driven criterion ever touches it.

3. Dropout masks activations during training and is not the same as permanently removing connections. Dropout zeroes activations only during training, sampling a different thinned network at every step; at test time every unit returns. Nothing is ever deleted - it is a temporary mechanism tied to the training process, not a structural edit. Permanently removing connections is different in kind: it leaves behind an actually smaller network, not a temporarily quiet one. There is more than one way to achieve that. The classical route trains a dense network first, then cuts weak connections after the fact and retrains what remains. A different route folds the removal into training itself: a sparsity-inducing regulariser can eliminate redundant con-

nections and neurons as a direct consequence of the optimisation, with no separate cutting step [@ma2019transformedell1regularizationlearning]. The lasso zeros in 4C sit closer to that second route - they fall out of the stationarity condition of a single convex fit, the soft-thresholding rule derived in Problem 3B, so they are a by-product of how the objective is solved rather than an edit applied after optimisation has finished. Either way, the outcome is nothing like dropout: dropout never permanently deletes anything, whereas here, once $\beta_j = 0$ at `lambda.min`, hidden unit j is out of the final model for good, with no "restore everything" step at prediction time.

4. One frozen hidden layer says nothing about a fully trained deep network. 4C differs from an actual deep network in two separate ways. The first is depth: 4C has exactly one nonlinear transformation, $x \to \text{ReLU}(Ax+c) \to \hat{y}$, while a deep network stacks several such transformations, each layer consuming the *learned* output of the one before it rather than a fixed random projection, so the composition can represent a far richer family of functions than any single fixed layer can. Concretely, a trained hidden layer could orient its ReLU boundaries toward whatever combinations of `lcavol`, `lcp`, and the other predictors actually separate high and low `lpsa`; the rows of our `A` instead point in directions drawn uniformly at random, with no relationship to `lpsa` at all, so most of them carry no more signal than noise. Stacking additional *trained* layers on top would let a real network re-combine those directions again and again, building increasingly task-specific features - something a single fixed layer cannot do regardless of how many hidden units it has. The second gap has nothing to do with depth: even that one layer is not learned here. A and c stay fixed at their randomly drawn values, and only β is fit, so the problem reduces to a linear model in the read-out weights. What matters is which parameters ever receive a gradient. In a trained network, the loss gradient flows backward through the hidden layer via backpropagation, telling every row of `A` how to rotate so that the resulting features better separate the signal in the data; in 4C, that gradient with respect to `A` is never computed or applied, so no such adaptation happens. It is worth noting that even training `A` on its own would only produce a *trained shallow* network, not a deep one - depth specifically requires composing several learned nonlinear transformations, and one adaptable layer is still just one layer. Because of both gaps, every result in 4C - including the fact that the nonlinear expansion barely moved the cross-validation error - is a statement about *one fixed, randomly oriented layer*. It says nothing about what a multi-layer network with a trained hidden representation could achieve on the same data; that would simply be a different experiment from the one run here.

Limitations.

(i) Sensitivity to the random-feature seed. The 199 hidden features depend entirely on the single seed 240402. `A` and `c` are only one draw among countless possible random weight sets, and nothing in this design lets us check whether this particular draw was more or less fortunate than another one would have been; the active-feature counts and CV-MSE in 4C could shift under a different seed. Fixing one seed is necessary for reproducibility, but it also means we cannot report how much these results would vary across draws.

(ii) A single, untrained random map confounds two explanations. Elastic net on the 5 original predictors reached a lower cross-validation MSE (Table 4B) than any model on the 199 ReLU features (Table 4C). Because `A` was drawn once and never adapted to `lpsa`, the experiment cannot separate two accounts of that gap: a nonlinear expansion is genuinely not useful for these predictors, or this *one* random, un-adapted expansion simply happened not to align with the signal. A trained hidden layer, or repeated draws of the random-feature bank, could in principle distinguish the two; this design cannot.

(iii) Small holdout. With 1202 training and only 301 test patients, the 4D holdout metrics carry

wide uncertainty; the final ranking can be confirmed only weakly, and gaps smaller than the cross-validation standard error should not be over-interpreted.

5 Problem 5. Report quality and reproducibility

5.1 5A. Reproducible submission

1. Open a clean R session.
2. Ensure the project uses the following relative directory structure:
   - `Group04_ProjectQuiz1.Rmd` (main report)
   - `data/prostate.csv` (original data)
3. Install the required packages if necessary:

```
install.packages(c("tidyverse", "glmnet", "broom", "knitr", "car"))
```

4. Render `Group04_ProjectQuiz1.Rmd`. Fixed random seeds (split: 240204, CV: 240304, features: 240404) guarantee reproducible train/test splits, cross-validation folds, and ReLU feature generation.
5. The holdout responses (`y_test`) remain isolated throughout data exploration, preprocessing, feature engineering, model fitting, and tuning. They first enter the computation in **Section 4D (Final Holdout Evaluation)** when calculating the final RMSE and MAE of the locked models.

5.2 5B. Statistical coummunication

5.2.1 Final Recommendation

Despite the OLS superiority in pure holdout accuracy, we recommend the lasso-dominant elastic net ($\alpha = 0.9$) for final clinical deployment.

Prediction vs. Interpretation:

- When strictly prioritizing prediction accuracy on this specific split, the OLS model is technically superior, achieving the lowest test RMSE (0.6713) and MAE (0.5736). However, OLS retains all eight candidate predictors leading to the complex model and OLS remains highly vulnerable to multicollinearity, such as assigning a counterintuitive negative coefficient to `lcp`.
- In contrast, prioritizing interpretation leads us to the elastic net model. It isolates the five most critical biological drivers of PSA levels (`lcavol`, `lweight`, `lbph`, `svi`, and `pgg45`) while shrinking noisy predictors to exactly zero (similar to Lasso). In a medical context, the interpretability, sparsity, and mathematical stability of this parsimonious model provide a much more practical diagnostic tool, easily justifying the modest trade-off in predictive error.

5.2.2 Limitations

1. Small Sample Size: The dataset contains only 97 observations in total, leaving roughly 20 patients in the holdout test set. This limited test sample makes the final evaluation highly sensitive to individual outliers, meaning the superior RMSE of the OLS model could partially result from test-set variance rather than true generalization ability.

2. Restricted Generalizability: The study population exclusively represents men with clinically localized prostate cancer who underwent radical prostatectomy. The models and their estimated coefficients therefore cannot be generalized to unscreened men in the general population.

3. Feature Space Complexity: As demonstrated in the nonlinear transformation experiment, projecting the clinical predictors into a high-dimensional space using fixed ReLU features overcomplicated the modeling task, introduced noise, and degraded holdout performance. This indicates that for this specific clinical dataset, simple linear assumptions are adequate, and complex neural-network-like architectures are unnecessary.

5.3 5C. AI verification record

AI-assisted item	How it was checked	Modification or rejection
Fixed R/R Markdown syntax and logic bugs in RStudio (ridge condition-number formula, broken inline R expressions in the lasso section, duplicated 3A/3B headers)	Document failed to compile before the fix; compiled successfully with correct output after applying it	Accepted the fix after independently recomputing the value and confirming it matched; rewrote the surrounding narrative ourselves
Grammar and phrasing checks on the narrative sections		Accepted wording-only fixes; rejected any suggestion that altered a numeric claim or added an unverified statement
Explained ridge/lasso theory (L2/L1 penalties, KKT conditions, soft-thresholding) to support the Problem 3 derivations		Used only as a study aid; the derivations and citations in 3A/3B are our own, referencing the primary sources directly
Explained the train-only, population-SD standardization approach (`fit_scaler`/`apply_scaler`) and why it prevents leakage		Adopted the explanation; kept the existing `fit_scaler` implementation unchanged

Preflight and session information

```r
# Add every object required to reproduce the final table and figures.
required_objects <- c(
  "train_id", "test_id", "x_train", "x_test", "y_train", "y_test", "foldid"
)
missing_objects <- setdiff(required_objects, ls())
if (length(missing_objects)) {
  stop(
    "Missing required objects: ",
    paste(missing_objects, collapse = ", ")
  )
}
stopifnot(length(intersect(train_id, test_id)) == 0L)
```

```r
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: x86_64-pc-linux-gnu
## Running under: Ubuntu 26.04 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
## LAPACK: /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblasp-r0.3.32.so;
 ↪  LAPACK version 3.12.0
##
## locale:
##  [1] LC_CTYPE=en_US.UTF-8       LC_NUMERIC=C
##  [3] LC_TIME=en_US.UTF-8        LC_COLLATE=en_US.UTF-8
##  [5] LC_MONETARY=en_US.UTF-8    LC_MESSAGES=en_US.UTF-8
##  [7] LC_PAPER=en_US.UTF-8       LC_NAME=C
##  [9] LC_ADDRESS=C               LC_TELEPHONE=C
## [11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C
##
## time zone: Asia/Ho_Chi_Minh
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] car_3.1-5       carData_3.0-6   knitr_1.51      broom_1.0.13
##  [5] glmnet_5.0      Matrix_1.7-4    lubridate_1.9.5 forcats_1.0.1
##  [9] stringr_1.6.0   dplyr_1.2.1     purrr_1.2.2     readr_2.2.0
## [13] tidyr_1.3.2     tibble_3.3.1    ggplot2_4.0.3   tidyverse_2.0.0
##
```

```
## loaded via a namespace (and not attached):
##  [1] generics_0.1.4     shape_1.4.6.1      stringi_1.8.7      lattice_0.22-9
##  [5] hms_1.1.4          digest_0.6.39      magrittr_2.0.5     timechange_0.4.0
##  [9] evaluate_1.0.5     grid_4.5.2         RColorBrewer_1.1-3 iterators_1.0.14
## [13] fastmap_1.2.0      foreach_1.5.2      backports_1.5.1    Formula_1.2-5
## [17] survival_3.8-6     scales_1.4.0       codetools_0.2-20   abind_1.4-8
## [21] cli_3.6.6          rlang_1.2.0        splines_4.5.2      withr_3.0.2
## [25] yaml_2.3.12        otel_0.2.0         tools_4.5.2        tzdb_0.5.0
## [29] vctrs_0.7.3        R6_2.6.1           lifecycle_1.0.5    pkgconfig_2.0.3
## [33] pillar_1.11.1      gtable_0.3.6       glue_1.8.1         Rcpp_1.1.2
## [37] xfun_0.58          tidyselect_1.2.1   rstudioapi_0.18.0  farver_2.1.2
## [41] htmltools_0.5.9    rmarkdown_2.31     compiler_4.5.2     S7_0.2.2
```